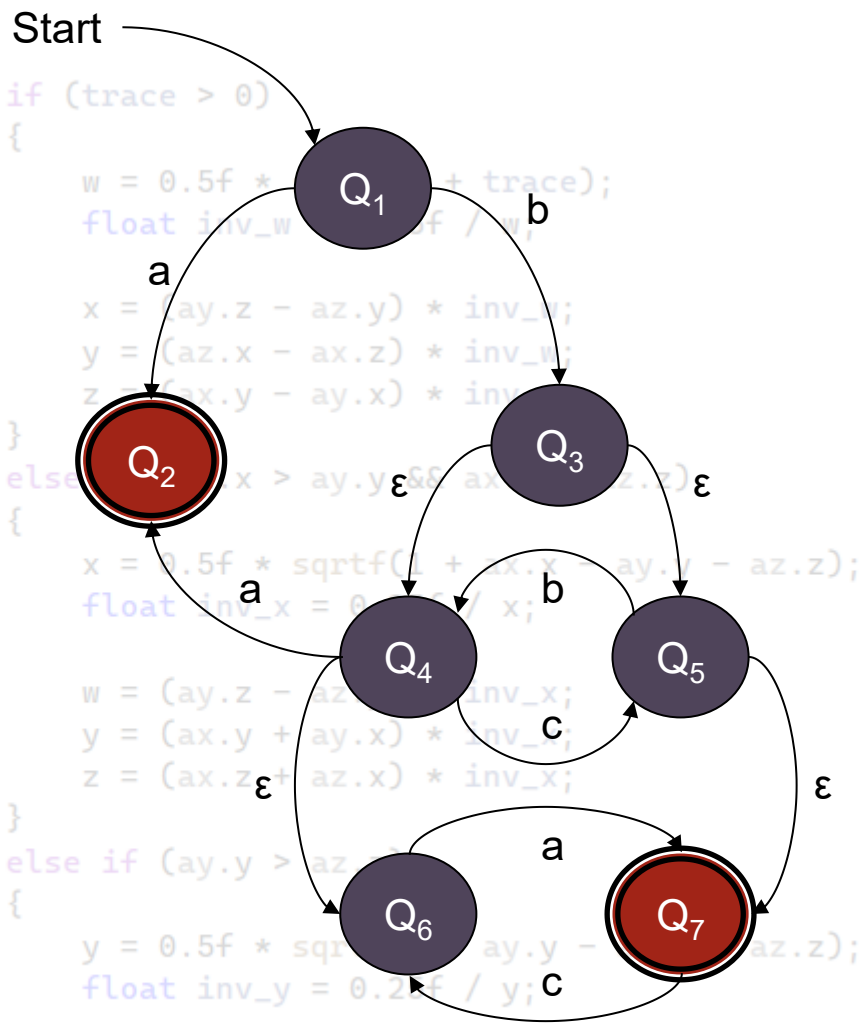
Algorithmen und Datenstrukturen

7 – Sortierende Container



Tobias Werner M. Sc.
Uni Bayreuth - Angewandte Informatik 3
Leibniz-FH Hannover
Duale Hochschule Sachsen
tobias.werner@uni-bayreuth.de

Einleitung

Heranführung

- Es ist dauerhaft ein sortierter Container von Daten zu pflegen,
  wobei regelmäßig Daten hinzukommen oder gelöscht werden.
  → Ist ein Array hier eine gute Idee?
  → Ist eine Liste hier eine gute Idee?

- Was ist der strukturelle Unterschied zwischen einer Palme,
  einer Tanne, einem Apfelbaum und einem Grashalm?

- Wann wird eine Tanne strukturell zum Grashalm?

Kompetenzen

- Daten in einem Baum verwalten: einfügen, abfragen, löschen
- selbst-ausbalancierende Bäume implementieren und bewerten

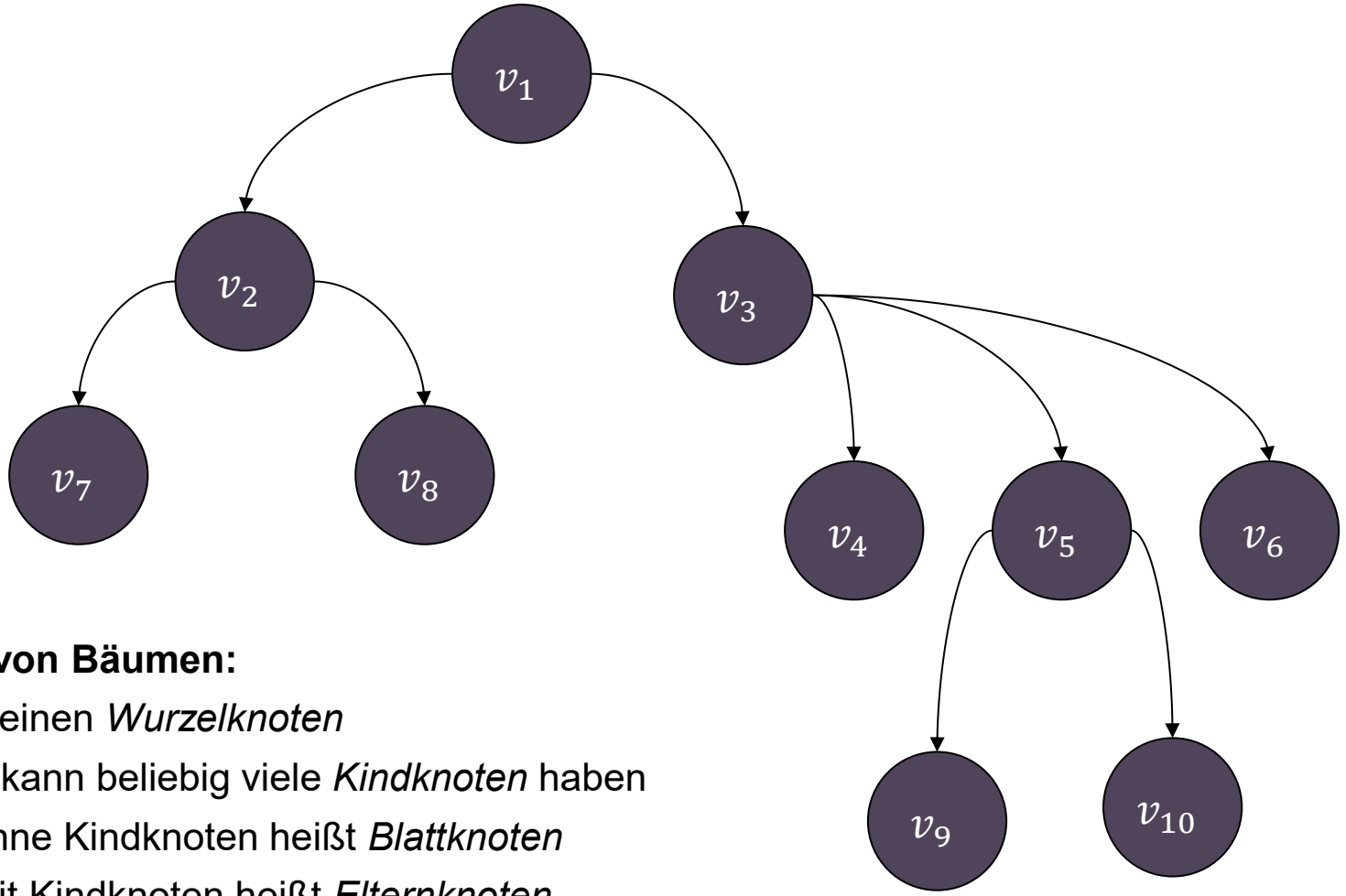
Vortragsübersicht

7 – Sortierende Container

- Bäume
- Breiten- und Tiefensuche
- AVL-Bäume
- Sets und Maps

Bäume (1)

Grundkonzept



Eigenschaften von Bäumen:

- es gibt exakt einen *Wurzelknoten*
- jeder Knoten kann beliebig viele *Kindknoten* haben
- ein Knoten ohne Kindknoten heißt *Blattknoten*
- ein Knoten mit Kindknoten heißt *Elternknoten*
- ein Elternknoten darf nicht (ggf. über andere Knoten) sein eigener Elternknoten sein

Bäume (2)

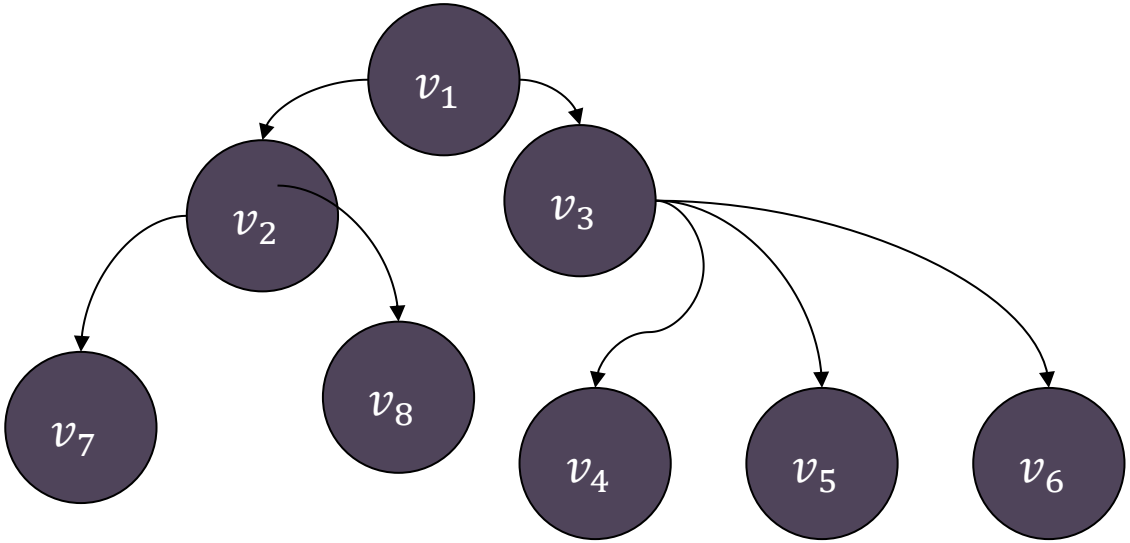
Formalisierung

ein Baum mit $n \in \mathbb{N}$ Knoten ist definiert über...

- ... eine Knotenmenge $V = \{v_1, v_2, \dots, v_n\}$,
- ... einen Wurzelknoten $v_\mathrm{R} \in V$ und
- ... eine Elternfunktion $p: V \to V \cup \emptyset$,

wobei...

- ... $p(v) = \emptyset \Leftrightarrow v = v_\mathrm{R}$ und
- ... $p\big(p(\dots p(v) \dots)\big) \neq v$ für beliebig viele, gültige Anwendungen von p.



$$V = \{v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8\}$$
$$v_\mathrm{R} = v_1$$

p mit folgender Wertetabelle:

v_1	v_2	v_3	v_4	v_5	v_6	v_7	v_8
$\emptyset$	v_1	v_1	v_3	v_3	v_3	v_2	v_2

Bäume (3)

Suchbäume

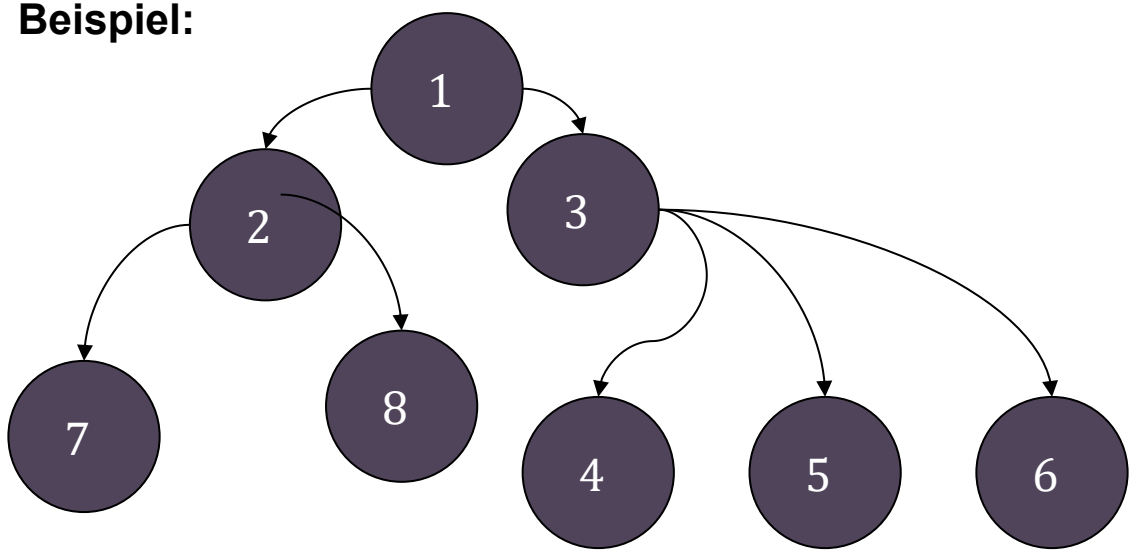
Idee: speichere zu jedem Knoten eine Zahl aus der Zahlenmenge Z ab
(z. B. über zusätzliche Zuordnungsvorschrift $f: V \to Z$)

einfache Umsetzung:

wähle $V \subset Z$

→ jeder Knoten entspricht einer im Baum vorhandenen Zahl

Beispiel:



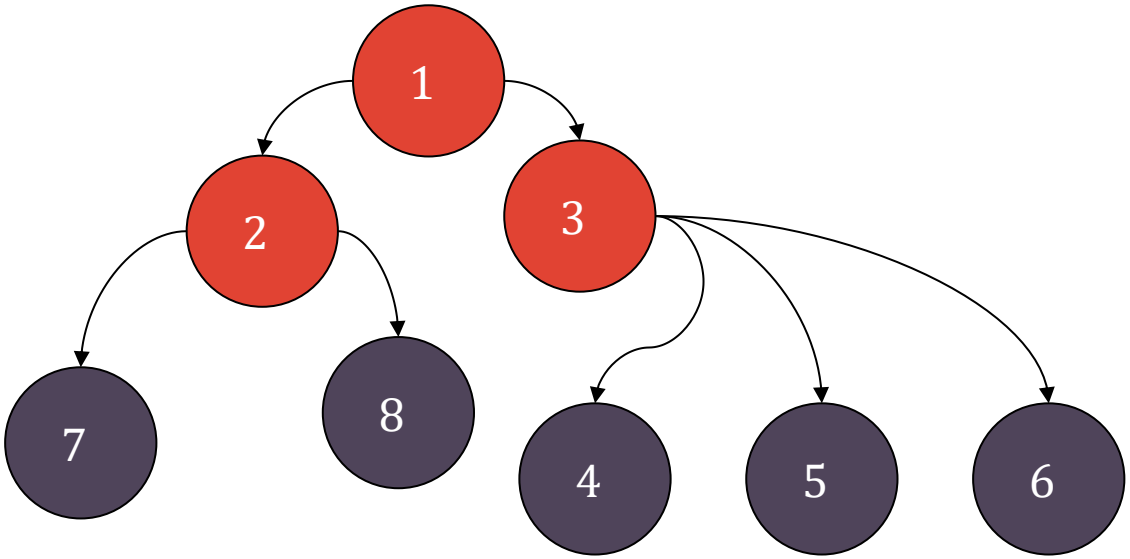
$$Z = \mathbb{Z}$$
$$V = \{\, 1, 2, 3, 4, 5, 6, 7, 8 \,\}$$

Bäume (4)

gegeben: Elternknoten v_{P} und dessen c Kinder $v_{\mathrm{P},1}, v_{\mathrm{P},2}. v_{\mathrm{P},3}, \dots, v_{\mathrm{P},c}$

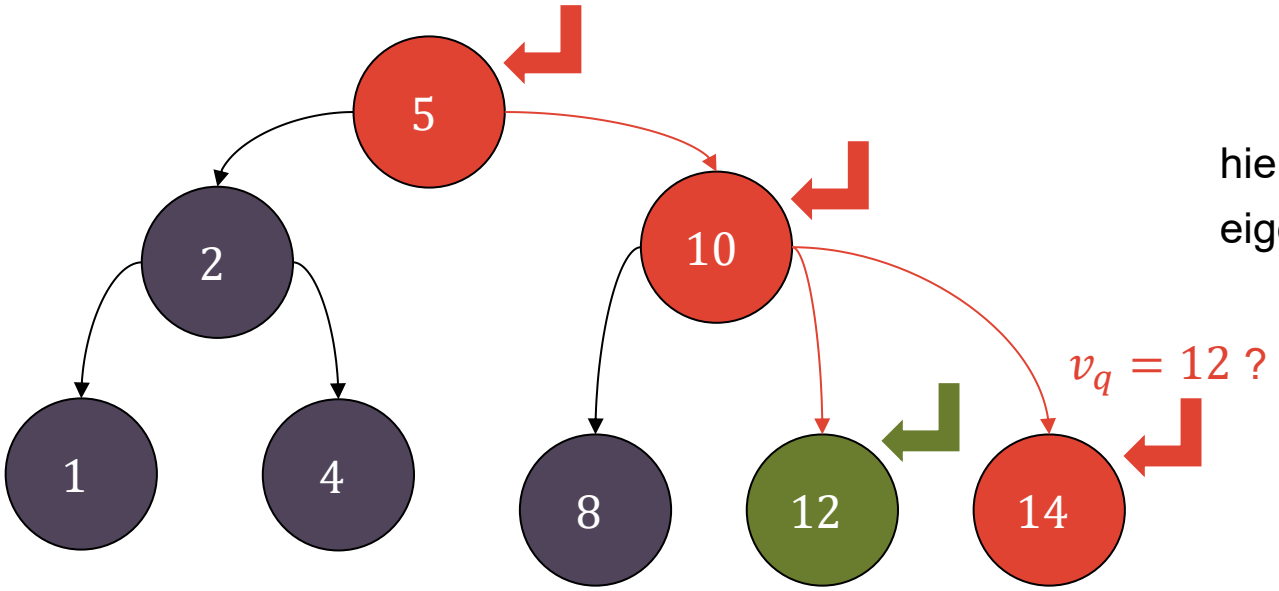
Definition *Suchbaumeigenschaft*:
zwischen jedem Elternknoten und den Werten in den Teilbäumen zu dessen Kindern besteht eine für den ganzen Baum gültige Ordnung, z. B. $v_{\mathrm{P},1} < v_{\mathrm{P}} < v_{\mathrm{P},3} < \cdots < v_{\mathrm{P},c}$.

Beispiele:



erfüllt die Suchbaum-eigenschaft nicht:
z. B. bei v_R ist $2 \not< 1$

Bäume (5)



hier ist die Suchbaum-eigenschaft erfüllt

Anwendung:

ist ein als Zahl gegebener Knoten $v_q \in V$?

→ prüfe nur diejenigen Äste, wo dieser Knoten sein kann

→ weil der Baum die Suchbaumeigenschaft erfüllt, ist diese Einschränkung möglich

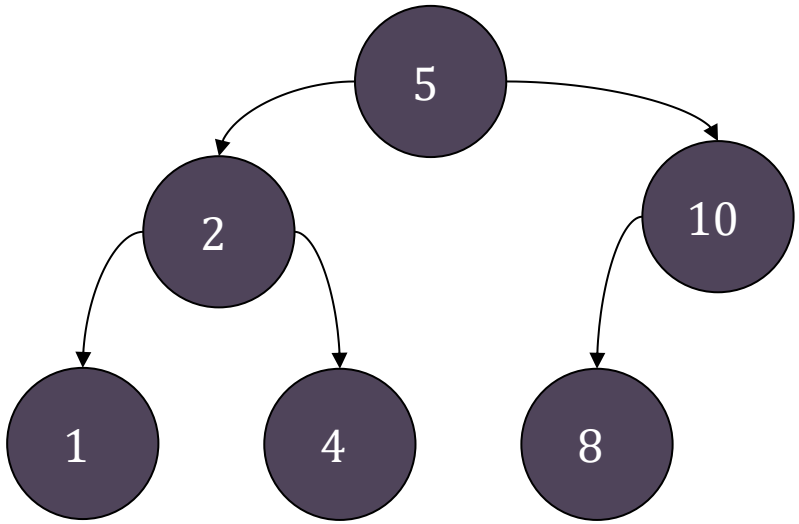
→ geht das noch geschickter?

Bäume (6)

Binäre Suchbäume

ein binärer Suchbaum ist ein Baum mit maximal zwei Kindern
pro Elternknoten,

- ... wobei die Werte im einen Kind (*das linke Kind*) und dessen weiteren Kindern immer kleiner als die Werte im Elternknoten sind und...
- ... wobei die Werte im anderen Kind (das rechte Kind) und dessen weiteren Kindern immer größer als die Werte im Elternknoten sind

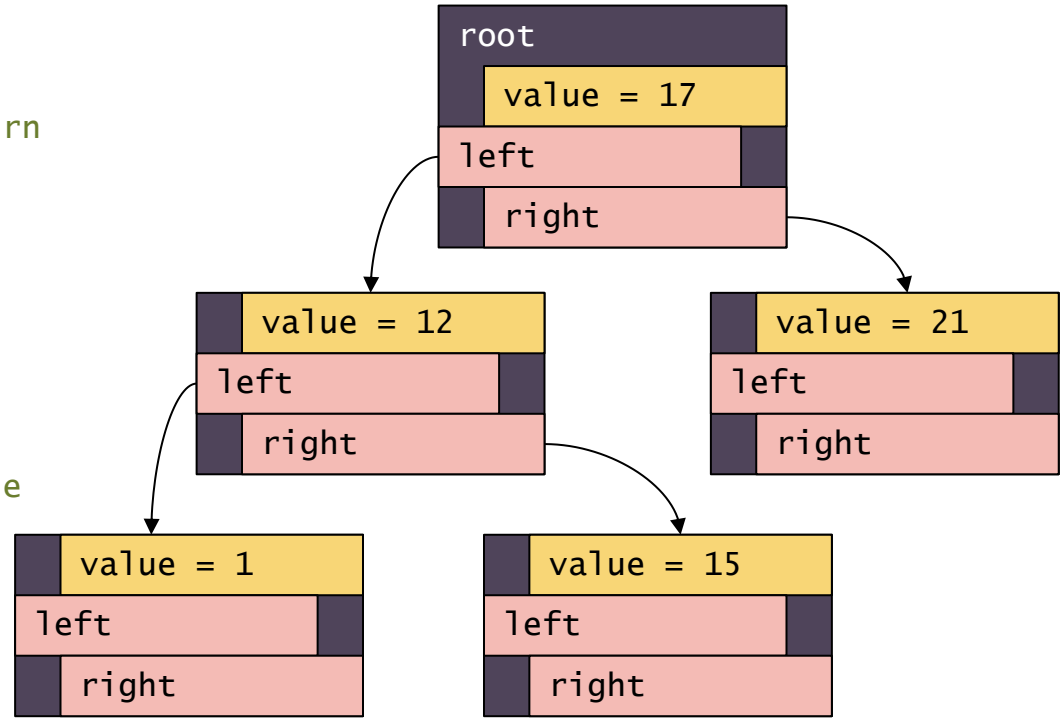


Bäume (7)

Einfügen in binären Suchbäumen

```
// ein Knoten mit Wert und Kinderzeigern
struct node
{
    node(V value) : value(value) { }
    V value;
    node *left;
    node *right;
};

// node ist der Knoten, unter dem Value
// eingefügt werden soll, anfangs der
// Wurzelknoten
void insert(V value, N* node)
{
    if (value < node->value)
    {
       if (node->left) insert(value, node->left)
          else node->left = new node(value);
    }
    else if (value > node->value)
    {
       if (node->right) insert(value, node->right)
          else node->right = new node(value);
    }
    // else: value ist im Baum schon enthalt
}
```



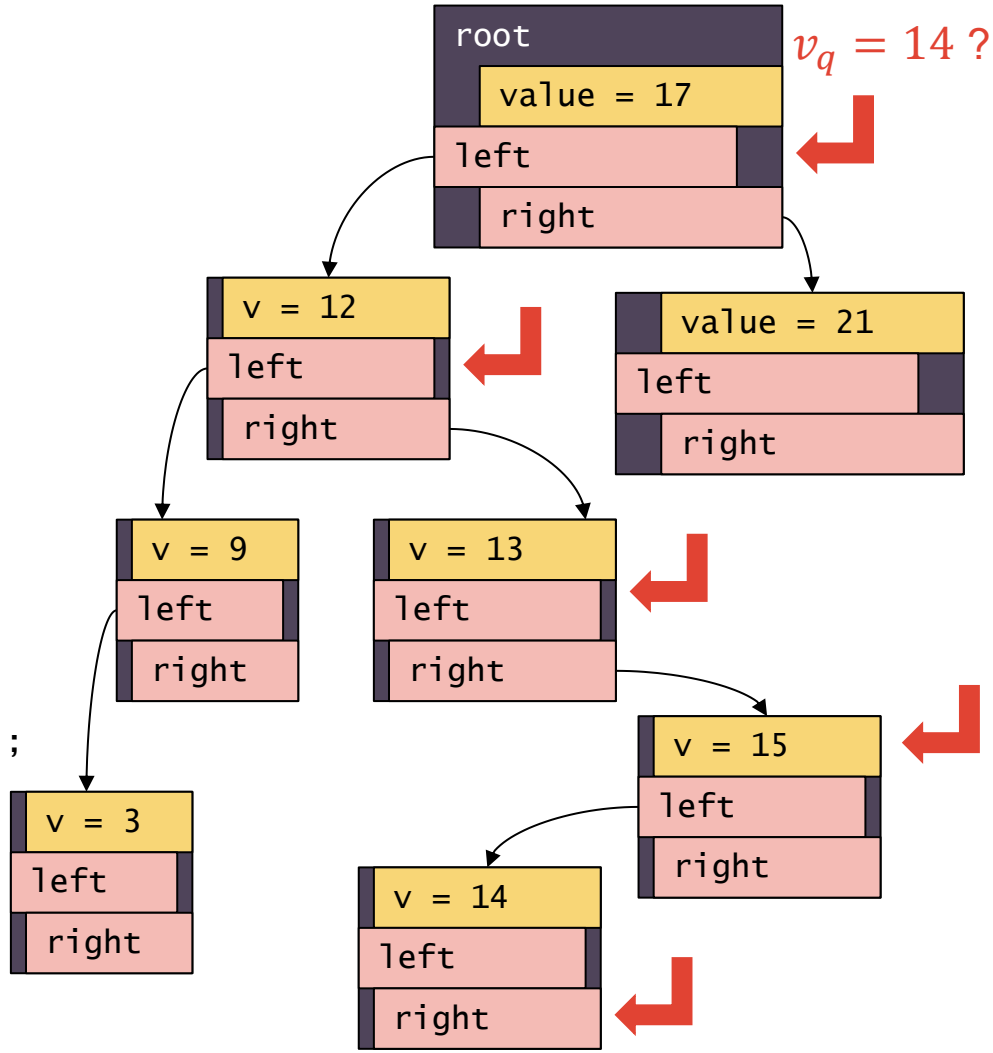
Bäume (8)

Suchen in binären Suchbäumen

```
// suche einen Knoten zu value
N* find(V value, N* node)
{
    // Wert muss im linken Teilbaum sein
    if (value < node->value)
    {
        if (node->left)
            return find(value, node->left);
        else
            return nullptr;
    }

    // Wert muss im rechten Teilbaum sein
    else if (node->value < value)
    {
        if (node->right)
            return find(value, node->right);
        else
           return nullptr;
    }

    // Wert ist gefunden!
    else
        return node;
}
```



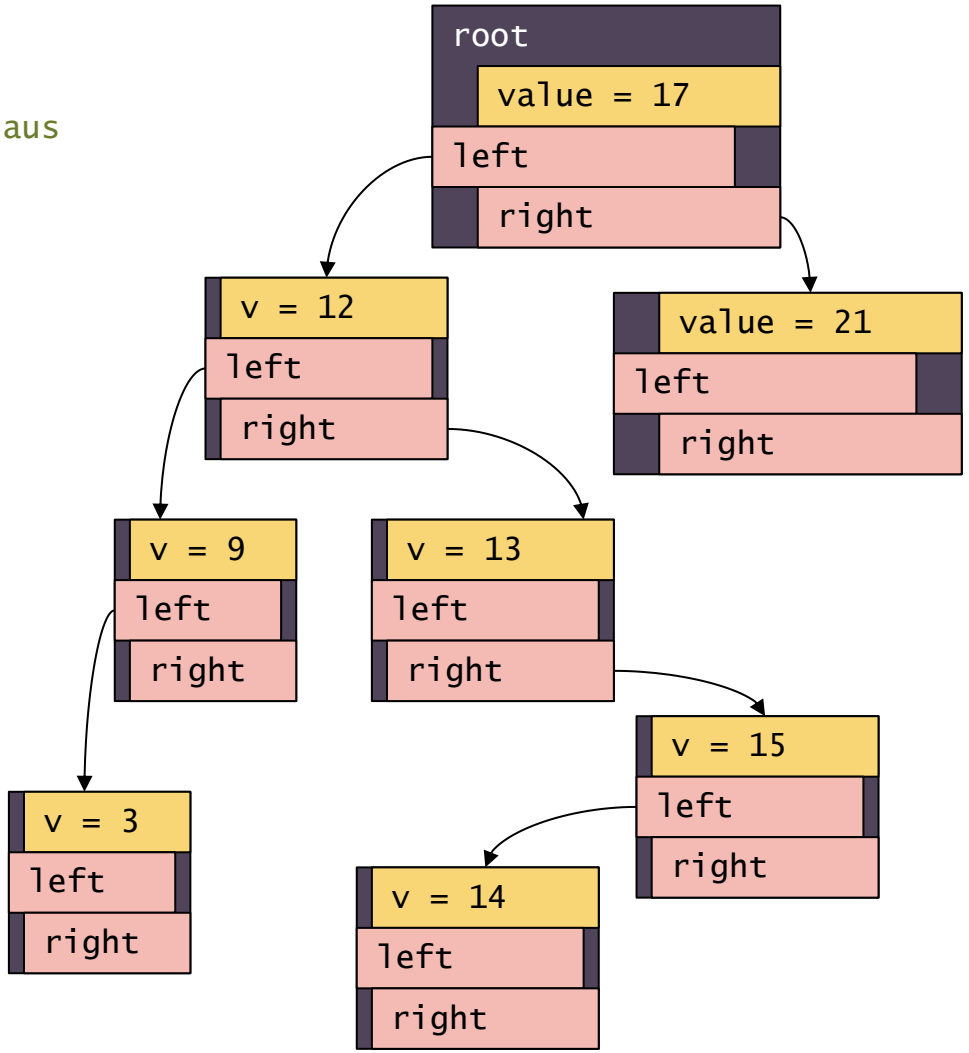
Bäume (9)

Nachbarn in binären Suchbäumen

```
// gib das nächst-kleinere Element zu node aus
N* next_smaller(N* node)
{
    // Nachar liegt direkt unter dem Knoten
    if (node->left)
    {
        N* result = node->left;
        while (result->right)
            result = result->right;
        return result;
    }

    // Nachbar liegt auf anderer Seite
    // von einem Elternknoten
    N* test = node;
    N* parent = node->parent;
    while (parent)
    {
        if (parent->right == test)
            return parent;
        test = parent;
        parent = parent->parent;
    }

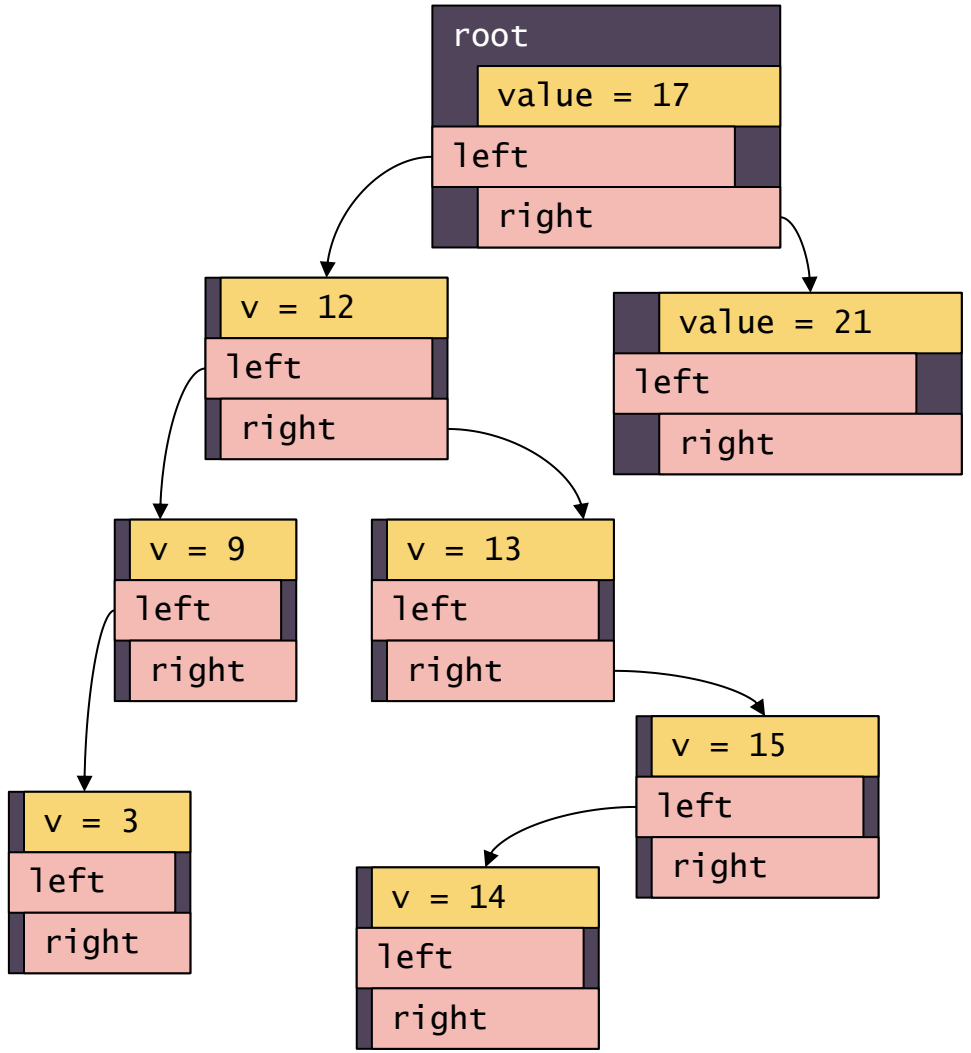
    return parent;
}
```



Bäume (10)

Löschen in binären Suchbäumen

```
void remove(node* root, int value)
{
    node* n = find(value);
    if (n) remove(n);
}
void remove(node* n)
{
    // löschen eines Blattes
    if (!n->left && !n->right)
    {
        if (n == root)
        {
            root = nullptr;
            delete n; --size;
        }
        else if (n->parent->left == p)
        {
            n->parent->left = nullptr;
            delete n; --size;
        }
        else if (n->parent->right == p)
        {
            n->parent->right = nullptr;
            delete n; --size;
        }
    }
```

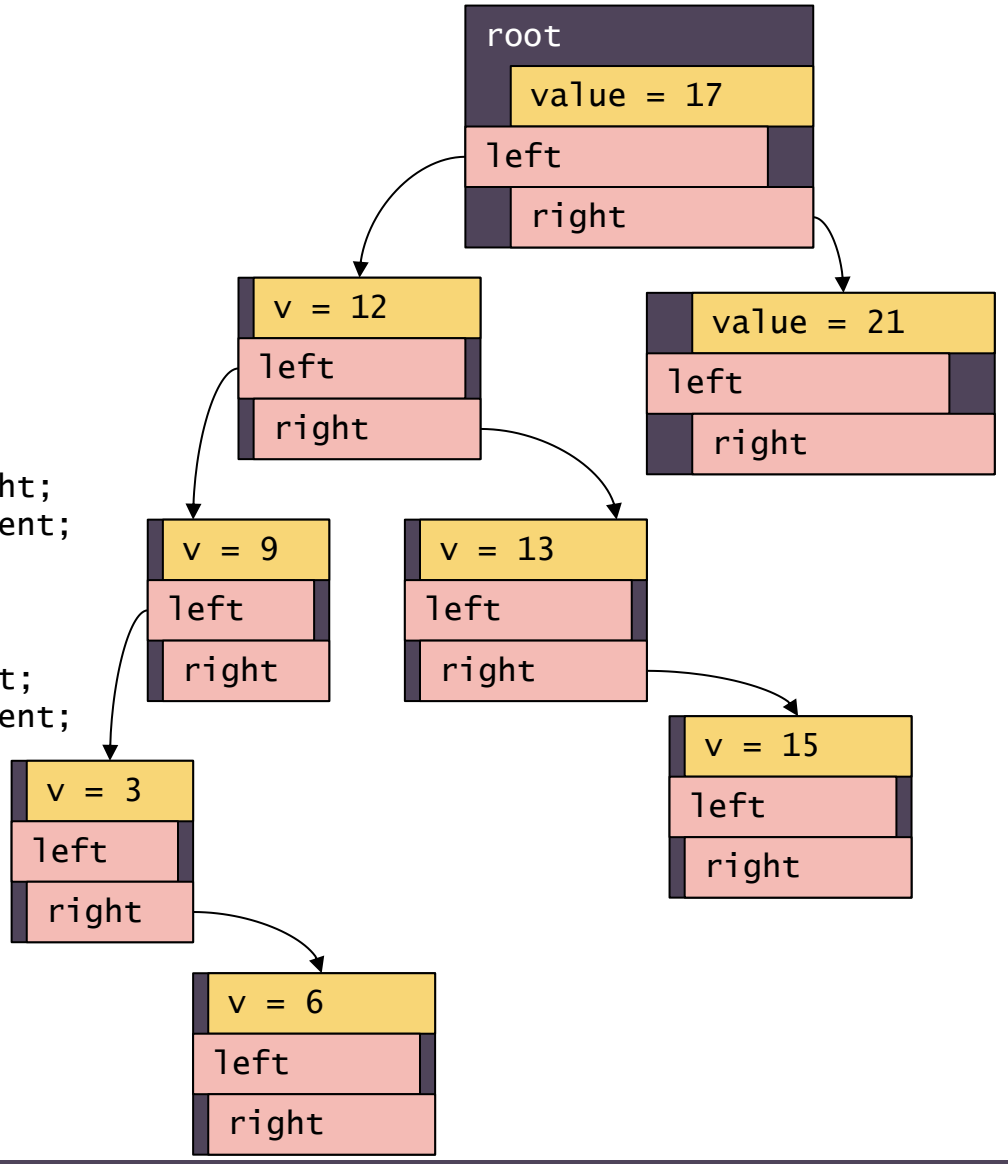


Bäume (11)

Löschen in binären Suchbäumen

```
// ...
// kein linkes Kind:
// kleinster Knoten in Ast
if (!n->left)
{
    if (n->parent)
    {
        if (n->parent->right == n)
        {
            n->parent->right = n->right;
            n->right->parent = n->parent;
        }
        else
        {
            n->parent->left = n->right;
            n->right->parent = n->parent;
        }

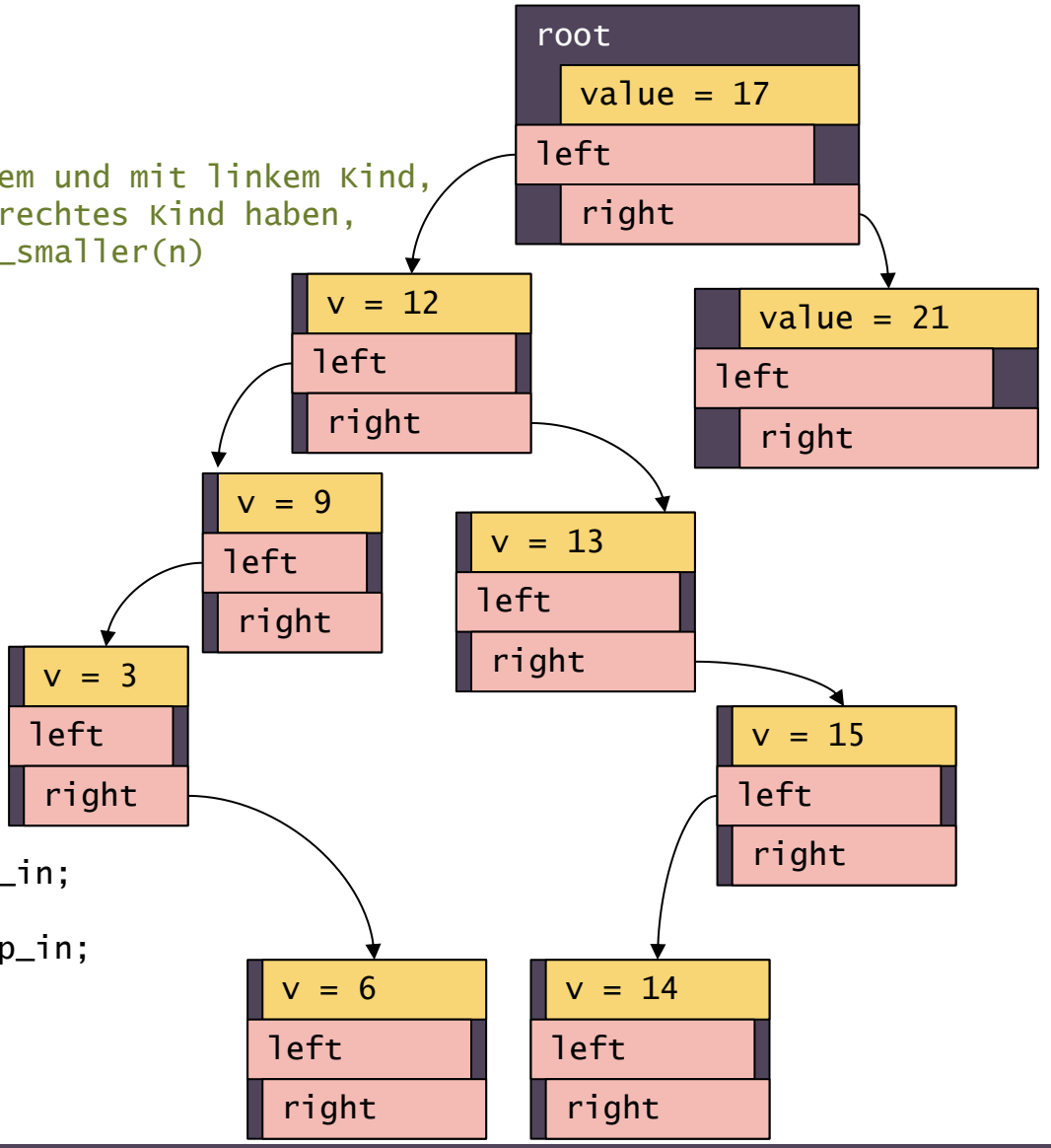
        delete n; --size;
    }
    else
    {
        root = n->right;
        n->right->parent = nullptr;
        delete n; --size;
    }
}
```



Bäume (12)

Löschen in binären Suchbäumen

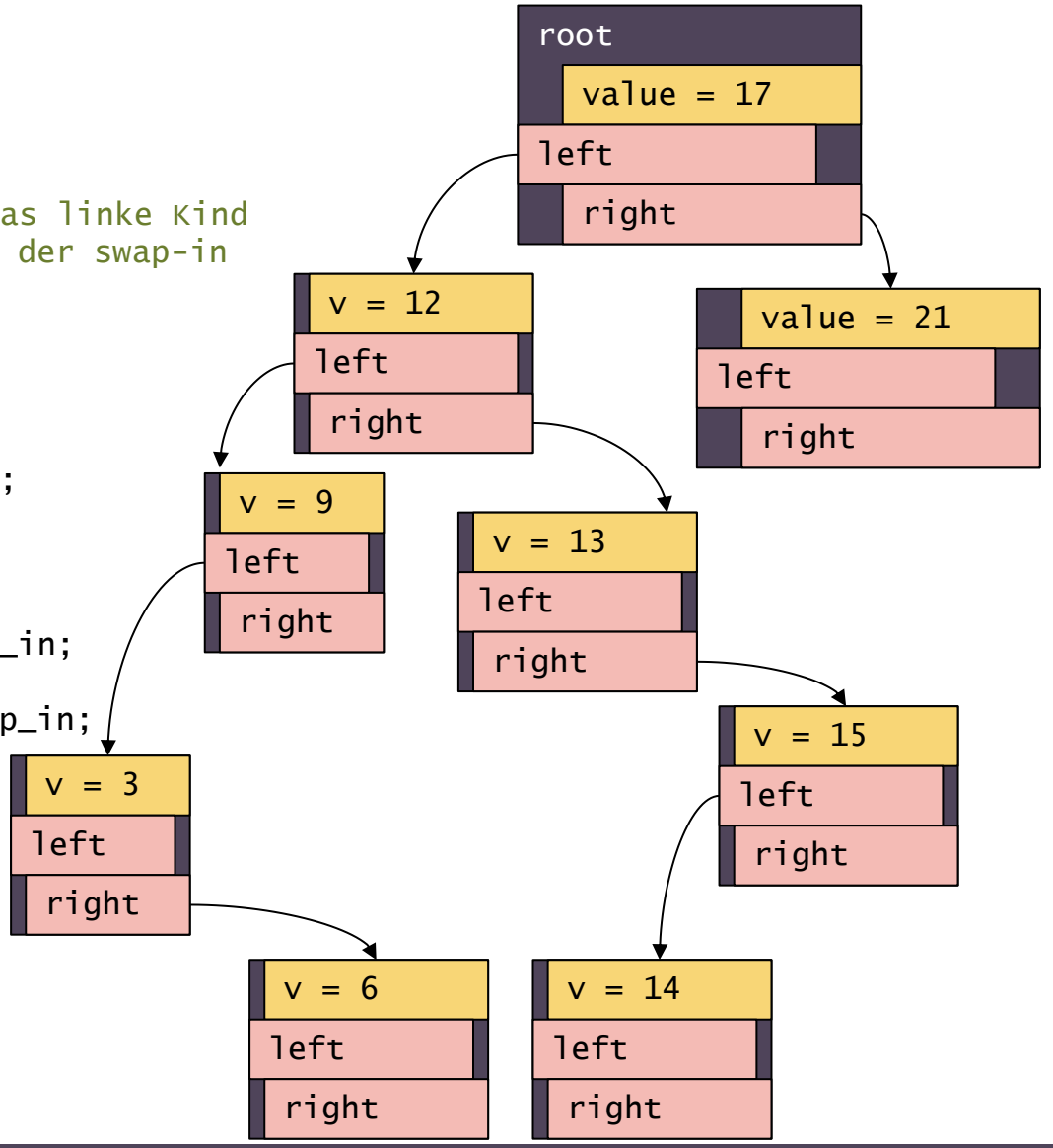
```
// ...
// Löschen eines Knotens mit rechtem und mit linkem Kind,
// Beobachtung: swap_in kann kein rechtes Kind haben,
// sonst wäre das rechte Kind next_smaller(n)
node* swap_in = next_smaller(n);
if (swap_in->parent != n)
{
    swap_in->parent->right =
        swap_in->left;
    if (swap_in->left)
        swap_in->left->parent =
            swap_in->parent;
    swap_in->left = n->left;
    swap_in->right = n->right;
    swap_in->parent = n->parent;
    if (n->parent)
    {
        if (n == n->parent->left)
            n->parent->left = swap_in;
        else
            n->parent->right = swap_in;
    }
    else root = swap_in;
    delete n; --size;
}
```



Bäume (13)

Löschen in binären Suchbäumen

```
        // ...
        // letzter Spezialfall:
        // der swap-in-Knoten ist direkt das linke Kind
        // vom zu löschenden Knoten, d. h. der swap-in
        // hat kein rechtes Knoten
        else
        {
            swap_in->right = n->right;
            if (n->right)
                n->right->parent = swap_in;
            if (n->parent)
            {
                if (n->parent->left = n)
                    n->parent->left = swap_in;
                else
                    n->parent->right = swap_in;
            }
            else root = swap_in;
            swap_in->parent = n->parent;
            delete n; --size;
        }
    }
```



Vortragsübersicht

7 – Sortierende Container

- Bäume
- Breiten- und Tiefensuche
- AVL-Bäume
- Sets und Maps

Breiten- und Tiefensuche (1)

Bäume traversieren

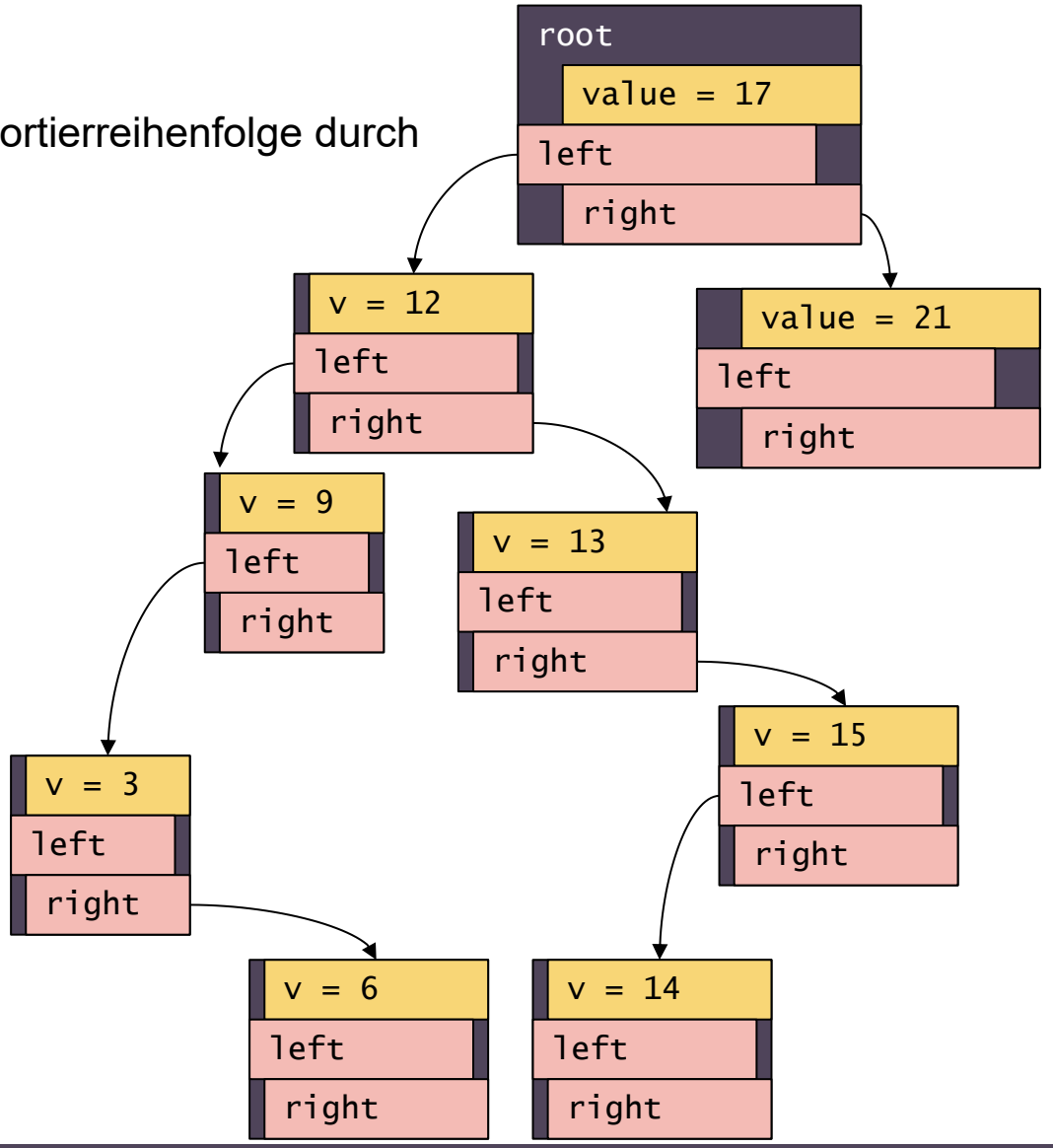
- *in-order-traversal*: gehe den Baum in Sortierreihenfolge durch

```
void print_in_order(tree& t)
{
    node* smallest = t.root;
    if (smallest == nullptr) return;
    while (smallest->left)
        smallest = smallest->left;
    for (node *n = smallest; n;
         n = next_greater(n))
        { std::cout << n->value; }

}
```

am Beispiel:

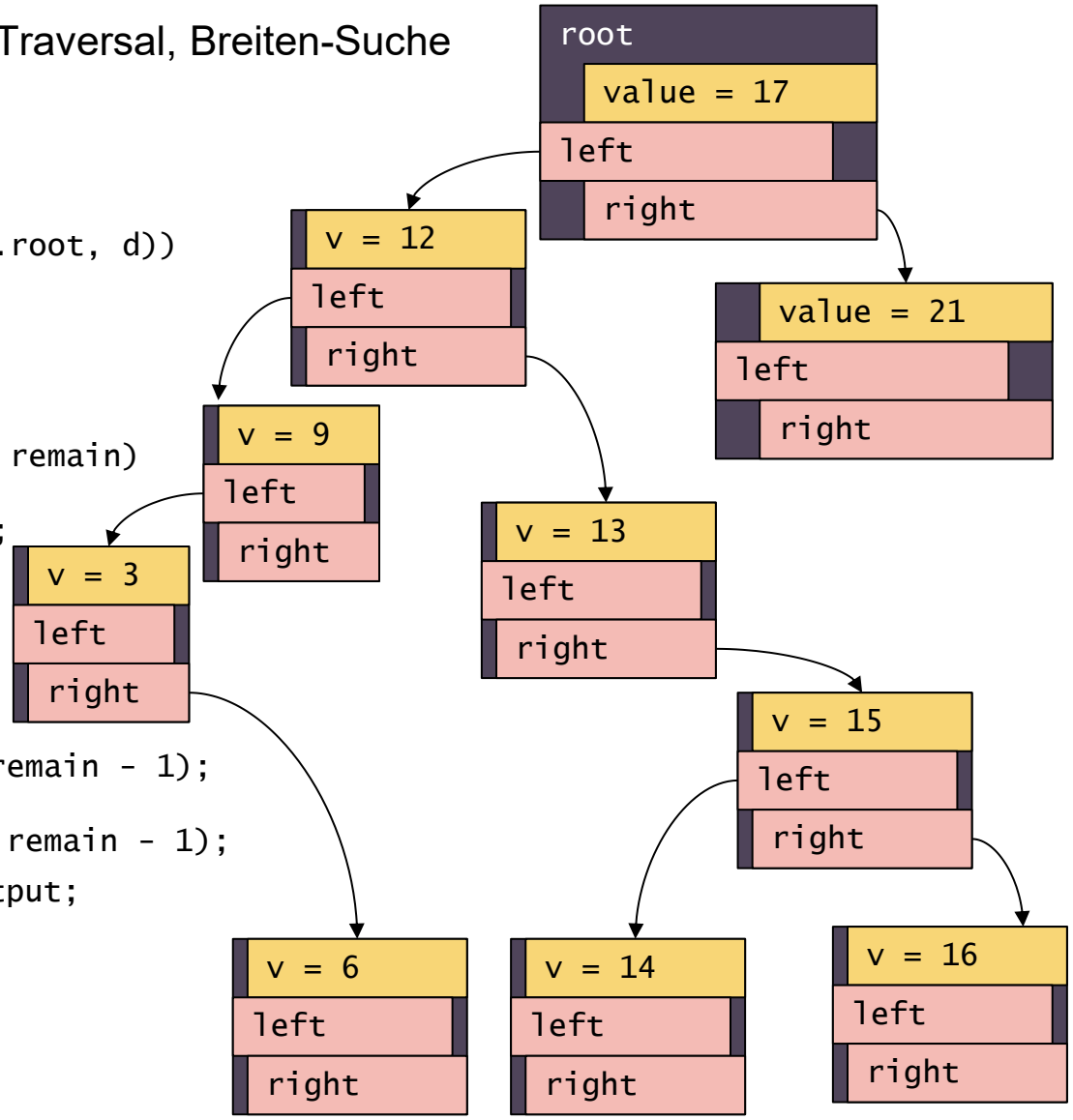
3, 6, 9, 12, 13, 14, 15, 17, 21



Breiten- und Tiefensuche (2)

- Breiten-Traversierung, Breadth-First-Traversal, Breiten-Suche

```
void traverse_breadth(tree& t)
{
    int d = 0;
    while (true) {
        if (!traverse_breadth(tree.root, d))
            break;
        ++d;
    }
}
bool traverse_breadth(node* n, int remain)
{
    if (n == nullptr) return false;
    if (remain == 0) {
        std::cout << n->value;
        return true;
    }
    bool left_output =
        traverse_breadth(n->left, remain - 1);
    bool right_output =
        traverse_breadth(n->right, remain - 1);
    return left_output || right_output;
}
```



Beispiel:
17, 12, 21, 9, 13, 3, 15, 6, 14, 16

Breiten- und Tiefensuche (3)

- Tiefen-Traversierung, Depth-First-Traversal, Tiefen-Suche

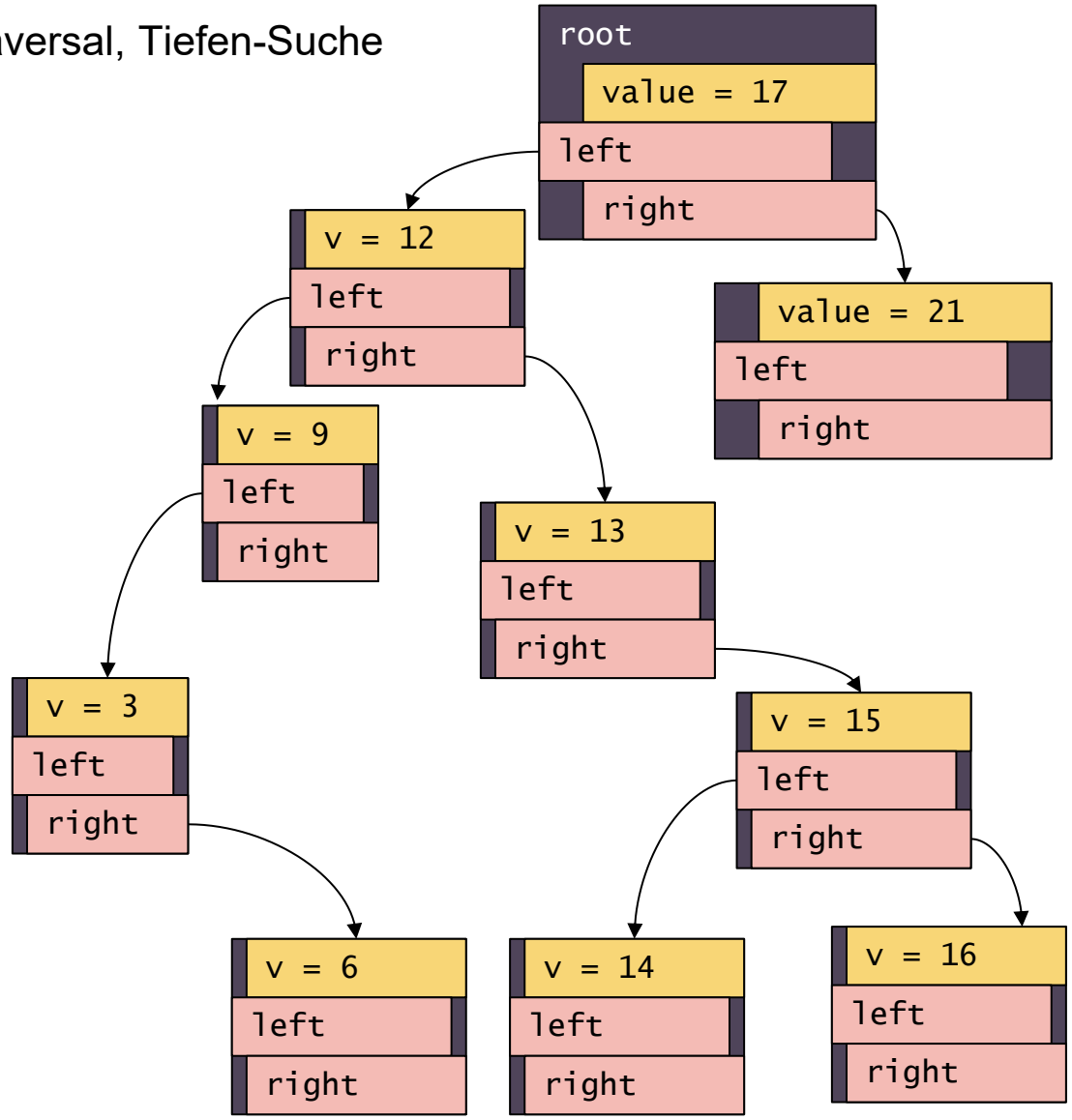
```
void traverse_depth(tree& t)
    { traverse_depth(t.root); }

void traverse_depth(node* n)
{
    if (n == nullptr) return;
    traverse_depth(n->left);
    traverse_depth(n->right);
    std::cout << n->value;
}
```

Beispiel:
6, 3, 9, 14, 16, 15, 13, 12, 21, 17

```
// alternativ:
void traverse_depth(node* n)
{
    if (n == nullptr) return;
    traverse_depth(n->left);
    std::cout << n->value;
    traverse_depth(n->right);
}
```

Beispiel:
3, 6, 9, 12, 13, 14, 15, 16, 17, 21



Vortragsübersicht

7 – Sortierende Container

- Bäume
- Breiten- und Tiefensuche
- AVL-Bäume
- Sets und Maps

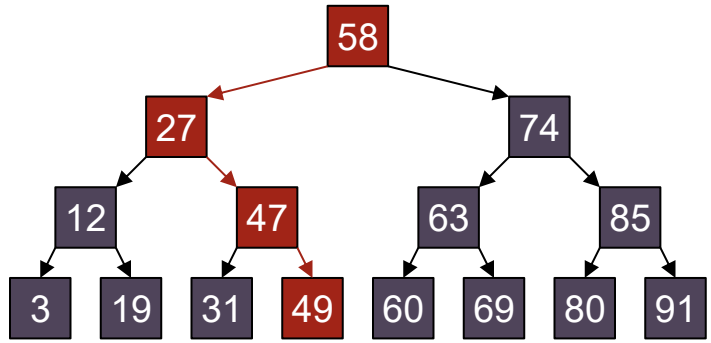
AVL-Bäume (1)

Suchproblem

typische Aufgabe:

suche in einem Container oft nach gegebenen Werten

(z. B. um Duplikate auszusortieren)



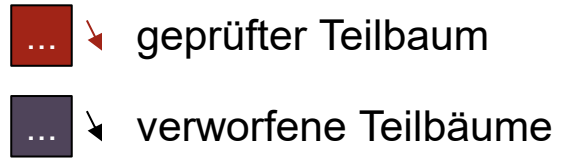
Lösungsansatz:

- baue vorab einen Suchbaum bekannter Werte
  → Kindknoten sind bezüglich dem Elternknoten sortiert

- steige bei der Suche durch den Baum ab
  → beginne am Wurzelknoten
  → der Wert kann sich nur in einem Teilbaum befinden
  → der andere Teilbaum muss nicht durchsucht werden

- Suchaufwand hängt von der Baumstruktur ab
  → Idealfall: Äste sind immer gleich lang, jede Verzweigung halbiert die Kandidatenzahl
  → Best-Case-Suchaufwand O(log *n*)

Beispiel:
Suche den Wert 52



AVL-Bäume (2)

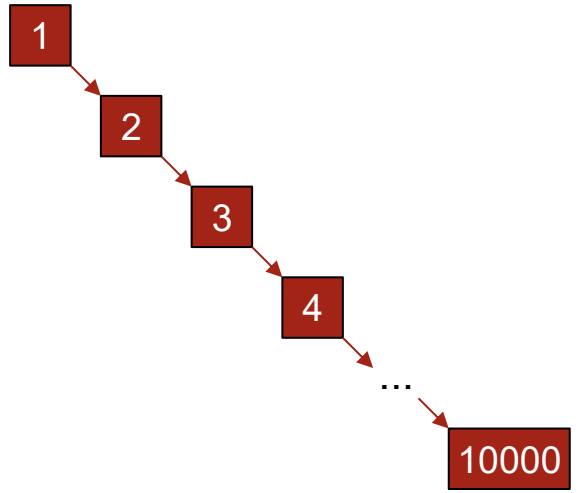
Negativbeispiel naives Einfügen

```
void tree_node::insert(int i)  // naive Implementierung
{
    i < value ?
        (left_ ? left_->insert(i) :
                 left_ = new tree_node(i)) :

    value < i ?
        (right_ ? right_->insert(i) :
                  right_ = new tree_node(i)) :

    return;
}

tree t;
for (int i = 0; i < 10000; ++i)
    t.insert(i);
t.find(10001);        // slow AF
```



Beispiel:
Suche den Wert 10001

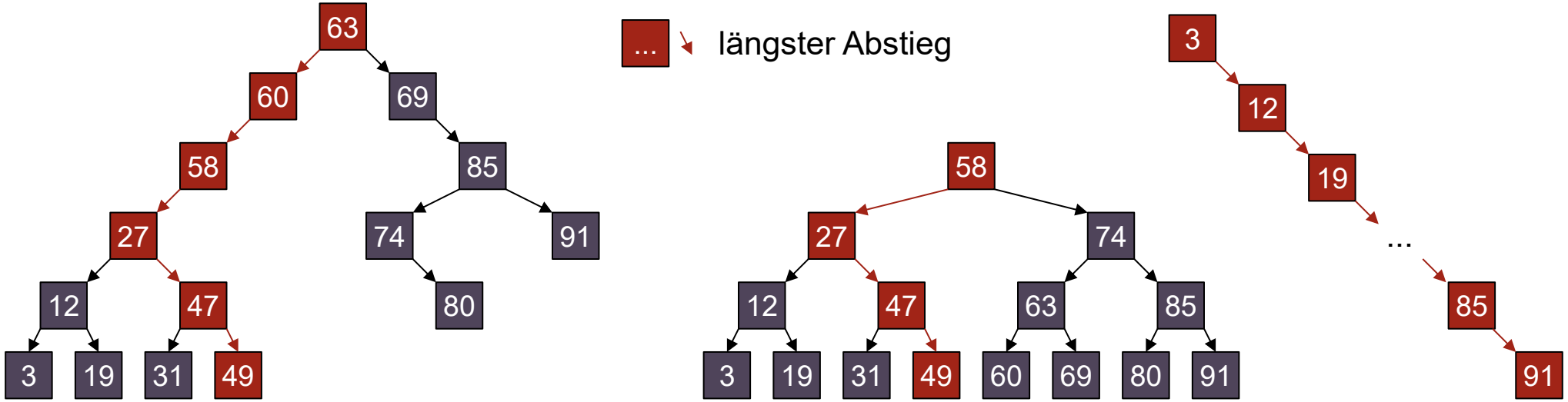
... geprüfter Teilbaum

- naive, binäre Such-Bäume *degenerieren* (z. B. Einfügen sortierter Daten)
  → *degenerierter Baum* ist "glorifizierte" Liste
  → Ausschluss von Teilbäumen nicht möglich
  → Worst-Case-Suchaufwand O(*n*)

AVL-Bäume (3)

Lösungsidee

- es gibt viele, mögliche Baumstrukturen für die selben Elemente
  → worst-case-Aufwand wird vom längstmöglichen Abstiegsweg bestimmt
- forme den Baum beim Einfügen und Löschen um, dass eine balancierte Struktur entsteht
  → beschränkte Differenz zwischen den Höhen von Teilbäumen
  → längstmöglicher Abstiegsweg ist nicht viel schlimmer als kürzester Abstiegsweg
  → worst-case-Suchaufwand O(log *n*)
  → *selbstbalancierende Bäume*, Praxis: *AVL-Bäume*, häufiger *Rot-Schwarz-Bäume*



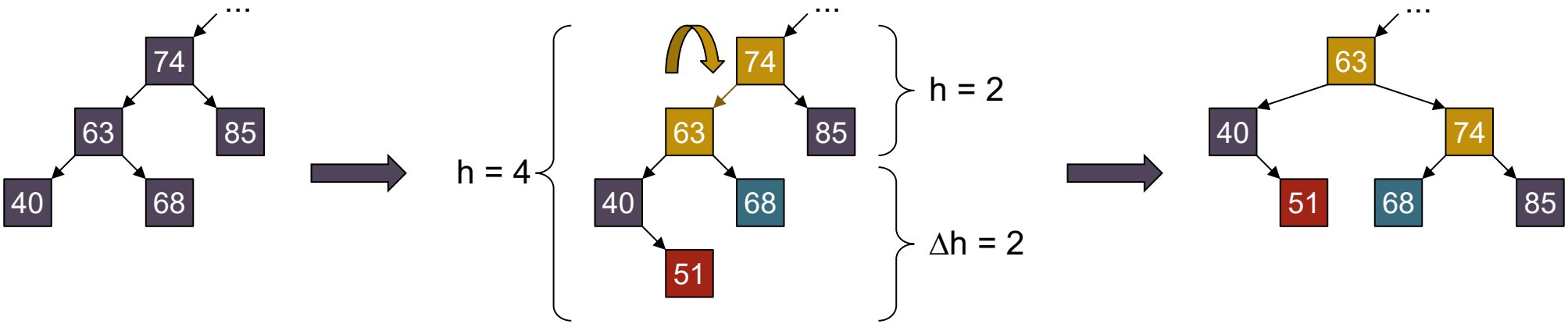
AVL-Bäume (4)

AVL-Bäume

(nach **A**delson-**W**elski und **L**andis, ca. 1960)

- AVL-Bäume vergleichen die Höhe von Teilbäumen nach Einfügen und Löschen
- Bei Höhendifferenz von 2 rotiert der AVL-Baum, um die Differenz auszugleichen

Kurzbeispiel:



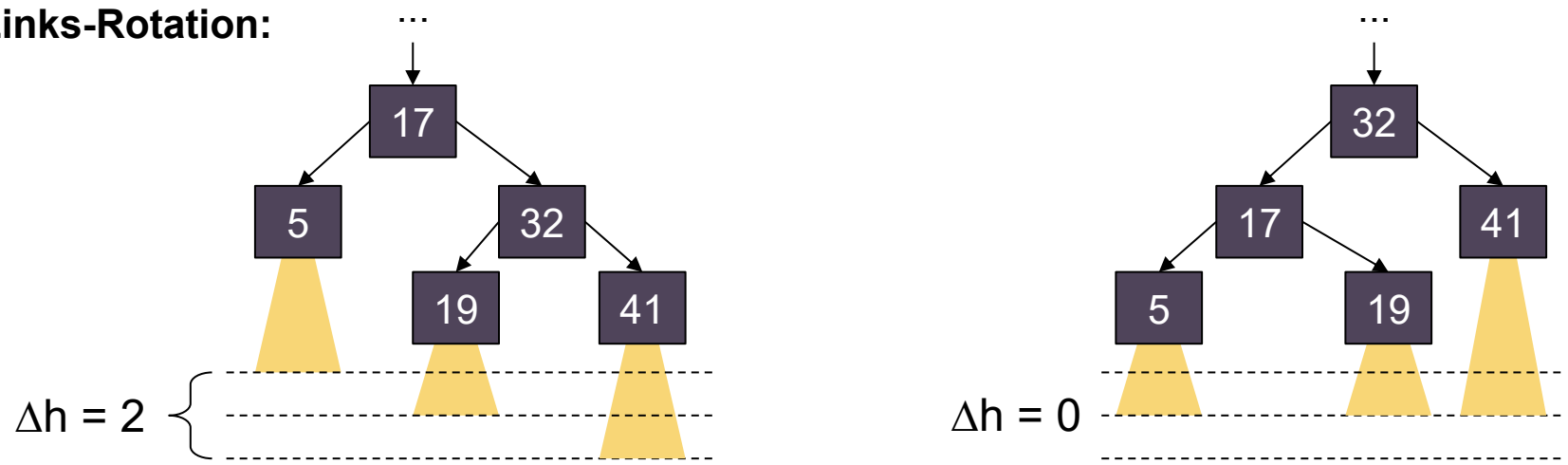
- Einfügen von 51 vergrößert Höhendifferenz Δh der Teilbäume von Knoten 74 auf $\Delta h = 2$
- Rotieren um 63↔74 gleicht Höhendifferenz aus
- Implementierung muss einige Spezialfälle berücksichtigen
  (z. B. Umhängen innerer Teilbäume, Doppelrotationen, Rotation an der Wurzel, ...)

AVL-Bäume (5)

Rotationen in AVL-Bäumen

- es gibt vier verschiedene Arten von Rotationen:
  Links-Rotation, *Rechts-Rotation*, *Links-Rechts-Rotation* und *Rechts-Links-Rotation*
  (die letzten beiden Rotationsvarianten nennt man auch *Doppelrotationen*)

Links-Rotation:



Vorbedingungen:

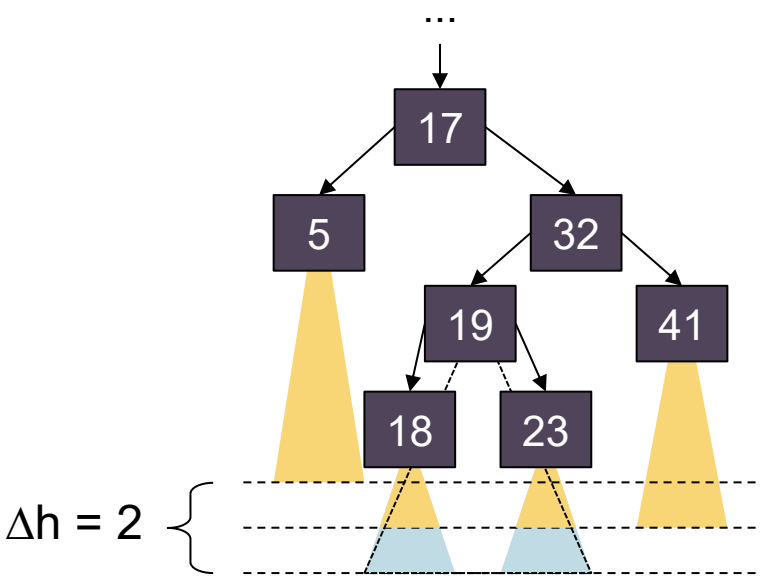
- der rechte Teilbaum hat eine um 2 größere Höhe
- der rechte Teilbaum vom rechten Kind ist um 1 höher als dessen linker Teilbaum

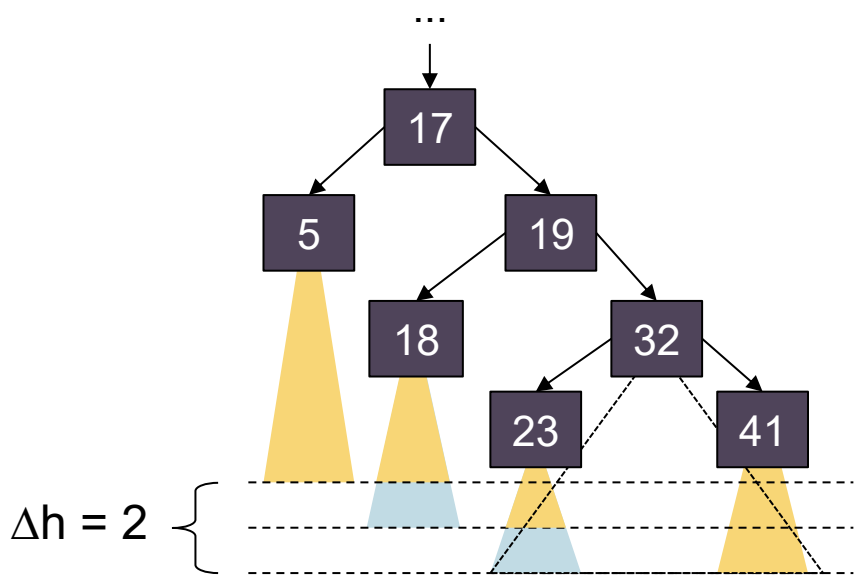
Operation:

ziehe den Elternknoten des rechten Teilbaums nach oben, den linken Teilbaum nach unten

AVL-Bäume (6)

Rechts-Links-Rotation:



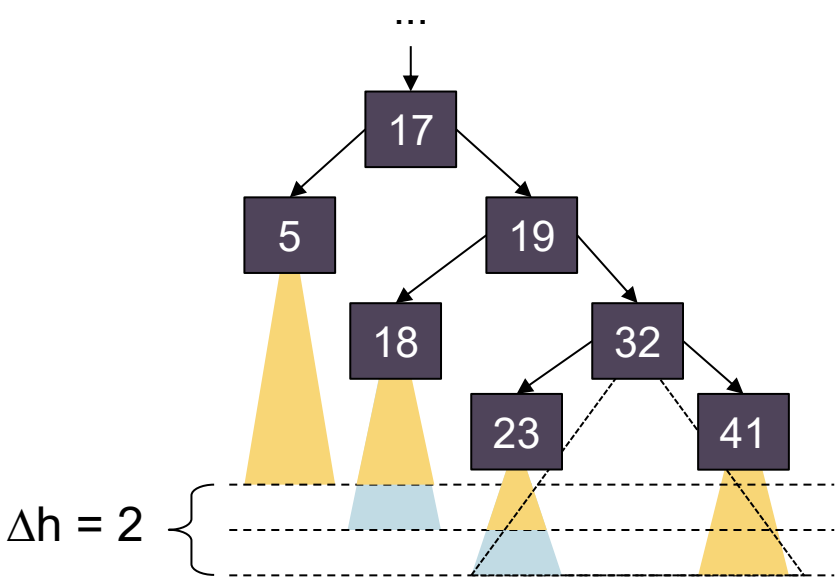


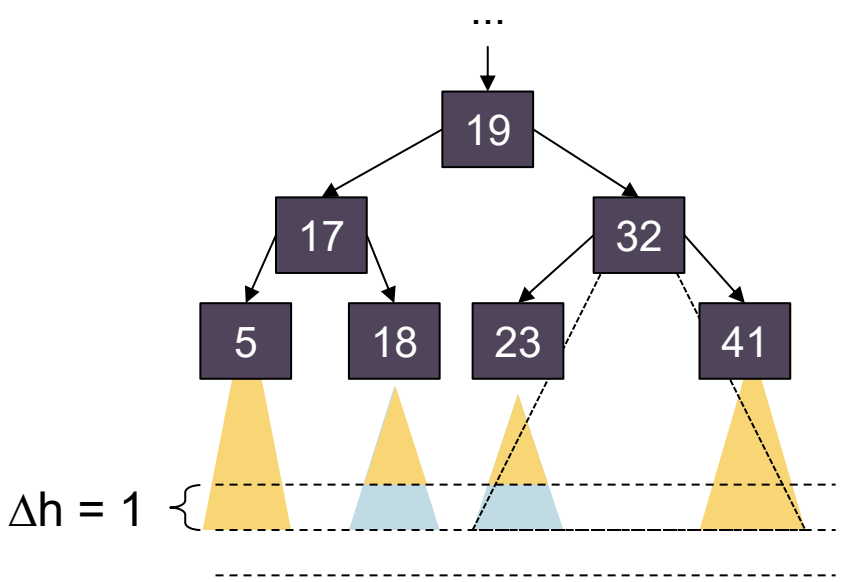
Vorbedingungen:

- der rechte Teilbaum hat eine um 2 größere Höhe
- der rechte Teilbaum vom rechten Kind ist um 1 niedriger als dessen linker Teilbaum

1. Operation ist Rechts-Rotation:
ziehe vom rechten Kind
den linken Teilbaum nach oben

AVL-Bäume (7)





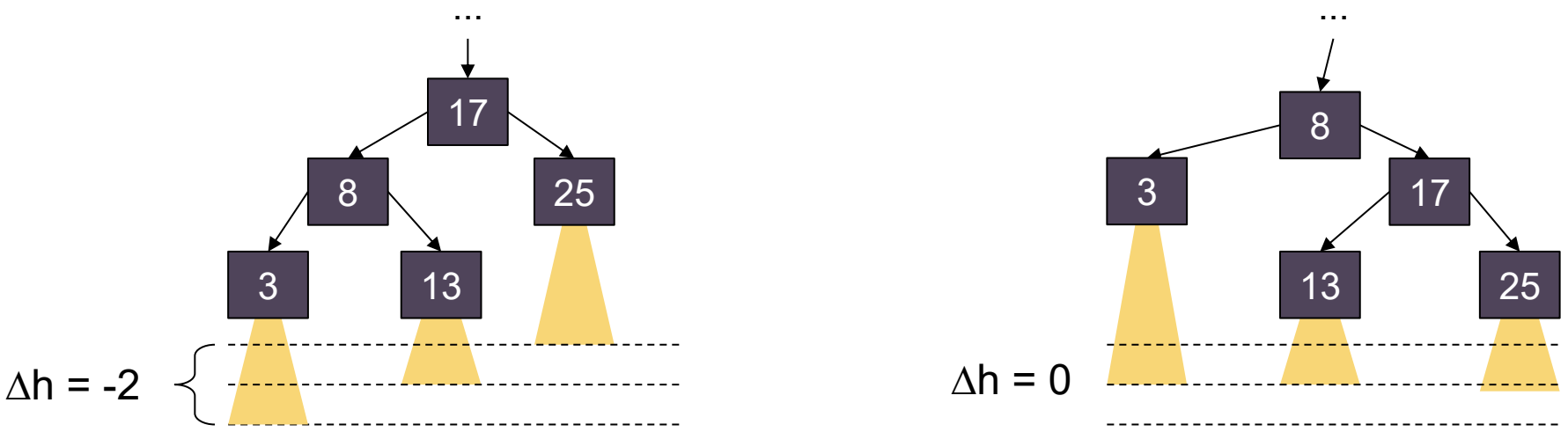
2. Operation ist Links-Rotation:
ziehe das rechte Kind nach oben,
das linke Kind nach unten

AVL-Eigenschaft:
die Differenz der Höhe $\Delta h = h_R - h_L$ aus der Höhe vom linken Teilbaum h_L
und der Höhe h_R vom rechten Teilbaum ist -1 (der linke Teilbaum ist höher),
0 (beide Teilbäume sind gleich hoch) oder 1 (der rechte Teilbaum ist höher)

→ die Rotationen stellen die AVL-Eigenschaft wieder her, wenn sie verletzt ist

AVL-Bäume (8)

Rechts-Rotation:



Vorbedingungen:

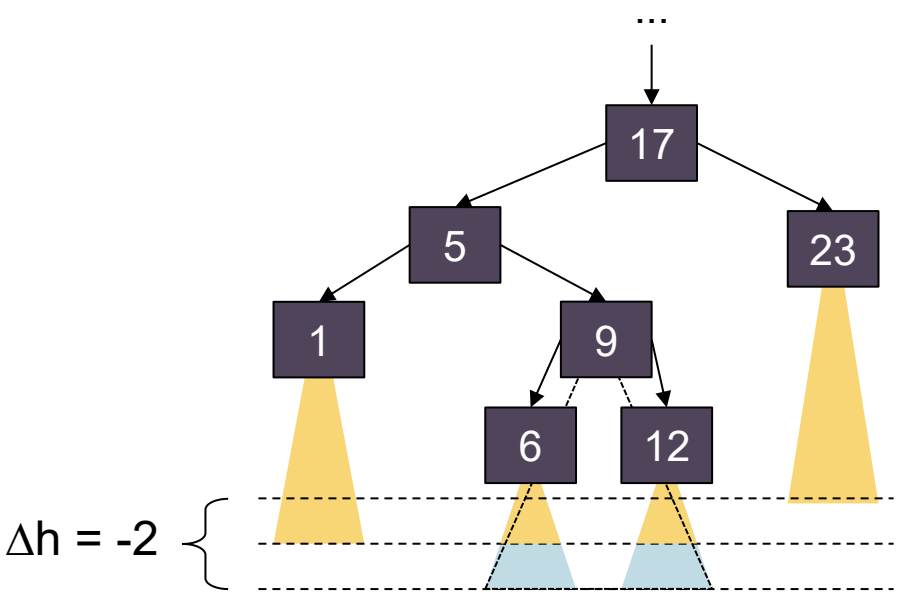
- der linke Teilbaum hat eine um 2 größere Höhe
- der linke Teilbaum vom linken Kind ist um 1 höher als dessen rechter Teilbaum

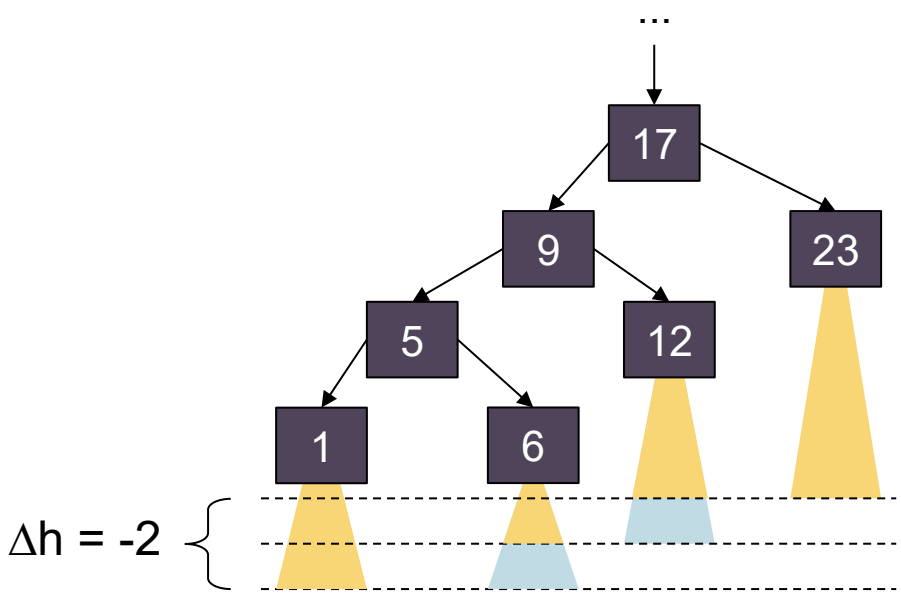
Operation:

ziehe den Elternknoten des linken Teilbaums nach oben, den rechten Teilbaum nach unten

AVL-Bäume (9)

Links-Rechts-Rotation:



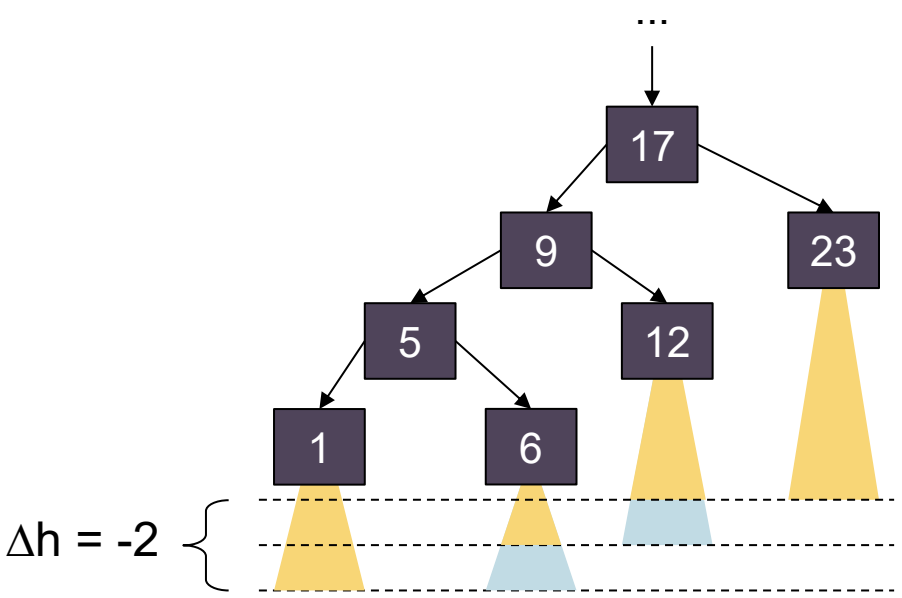


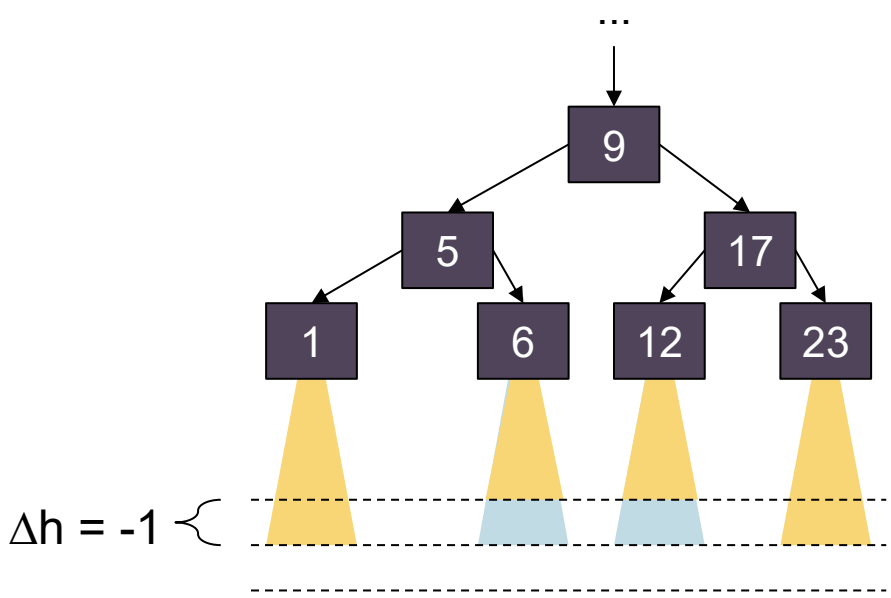
Vorbedingungen:

- der linke Teilbaum hat eine um 2 größere Höhe
- der linke Teilbaum vom linke Kind ist um 1 niedriger als dessen rechter Teilbaum

1. Operation ist Links-Rotation:
ziehe vom linken Kind
den rechten Teilbaum nach oben

AVL-Bäume (10)





2. Operation ist Rechts-Rotation:
ziehe das linke Kind nach oben,
das rechte Kind nach unten

Beobachtung:

die Höhendifferenz erreicht höchstens Werte zwischen -2 und +2

→ speichere statt der Höhe der Teilbäume die Höhendifferenz als *Balance-Wert*

→ der Balance-Wert lässt sich nach dem Einfügen in einen AVL-Baum aktualisieren

AVL-Bäume (11)

Gesamtalgorithmus zum Einfügen in einen AVL-Baum:

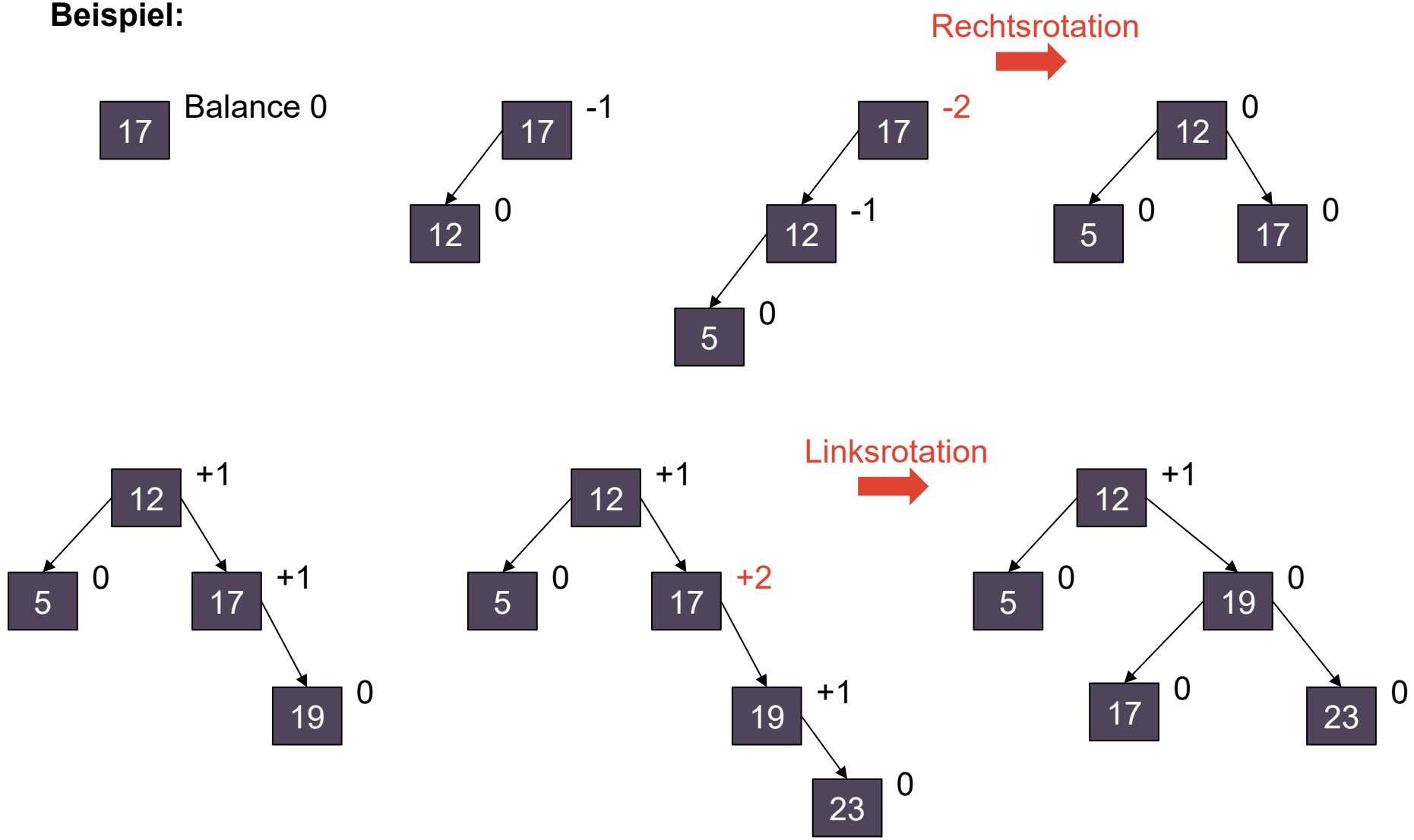
1. finde einen Elternknoten `p` für den einzufügenden Wert `v`
2. falls `p.v == v`, ist der Wert schon im Baum vorhanden, Abbruch

3. füge `v` als neuen Knoten rechts oder links von `p` ein
   → falls wir links von `p` eingefügt haben: rechne `p.balance -= 1`
   → falls wir rechts von `p` eingefügt haben: rechne `p.balance += 1`

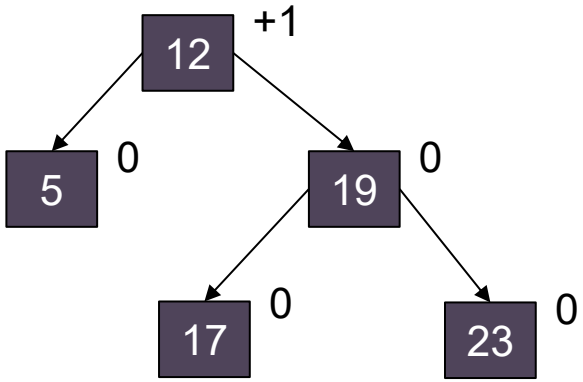
4. falls sich die `balance` von `p` auf `+1` oder `-1` verändert hat:
   → setze `gp = p.parent` (anfangs der Großvater des eingefügten Knotens)
   → falls `gp == nullptr` Ende
   → falls `p` das linke Kind von `gp` war, setze `gp.balance -= 1`
   → falls `p` das rechte Kind von `gp` war, setze `gp.balance += 1`
   → falls `gp.balance` den Wert `+2` oder `-2` erreicht hat:
   Rotieren des Baums (vgl. vorher), um `gp.balance` in Richtung 0 zu verändern,
   passe alle `balance`-Werte entsprechend an (ggf. von Kindern von `gp`)
   → setze `p = gp`

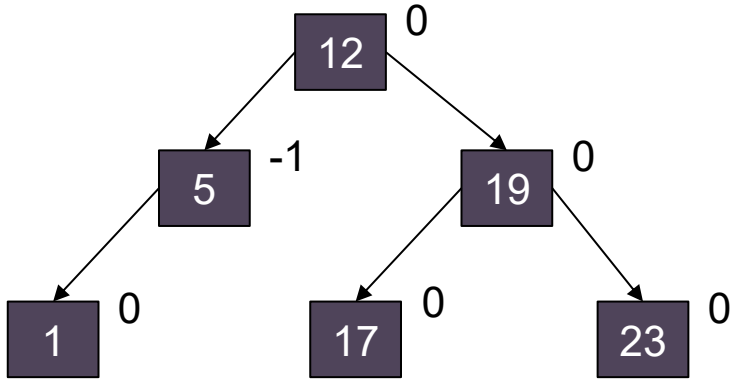
AVL-Bäume (12)

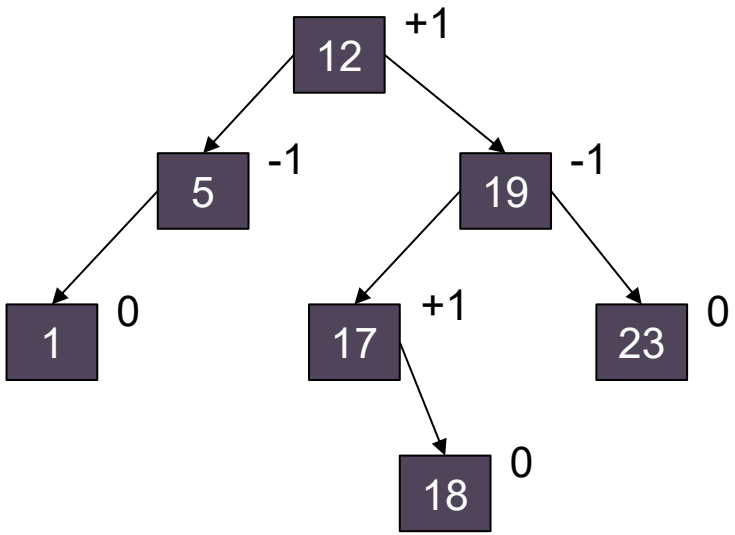
Beispiel:

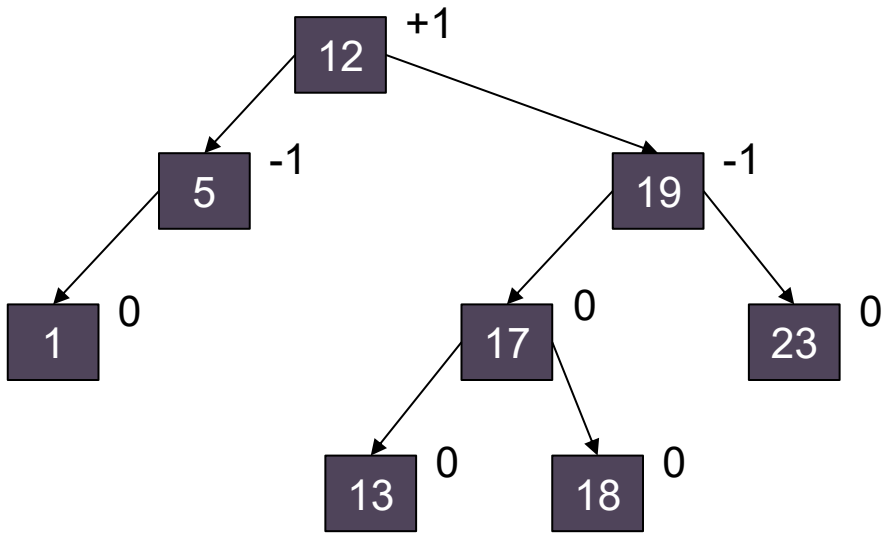


AVL-Bäume (13)

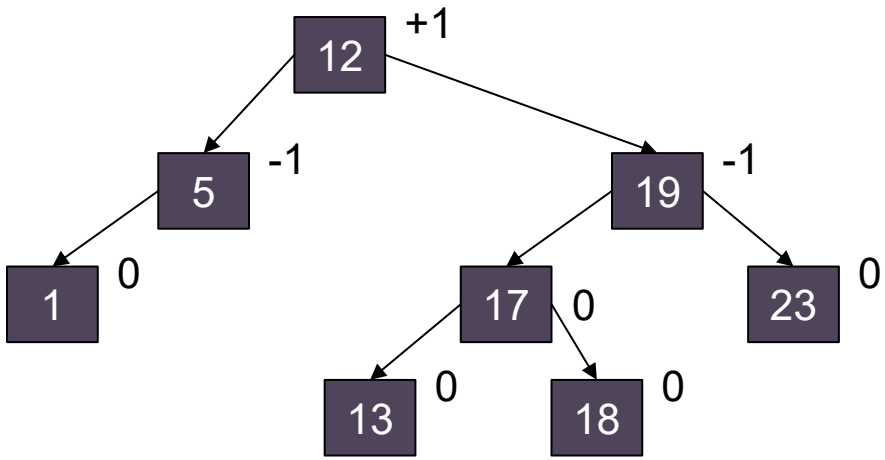

12
+1
5
0
19
0
17
0
23
0

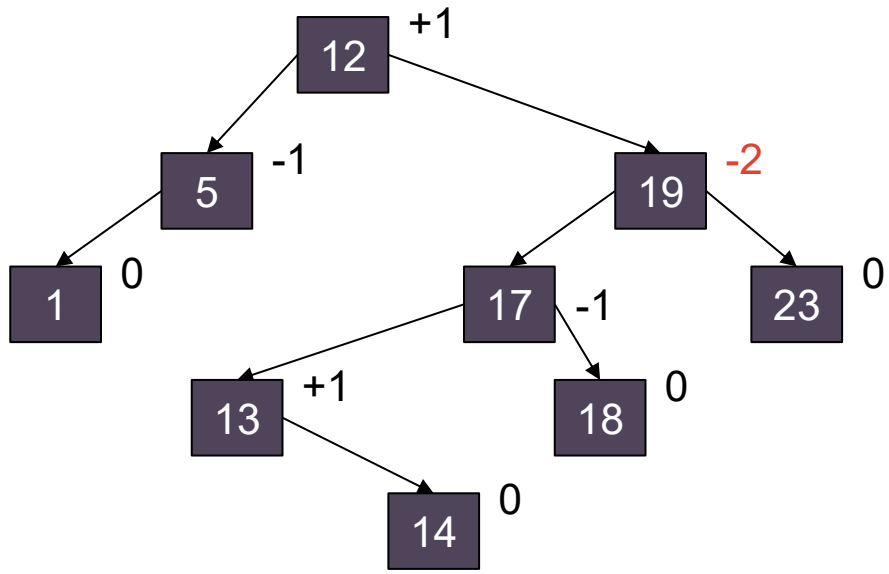

12
0
5
-1
19
0
1
0
17
0
23
0


12
+1
5
-1
19
-1
1
0
17
+1
23
0
18
0

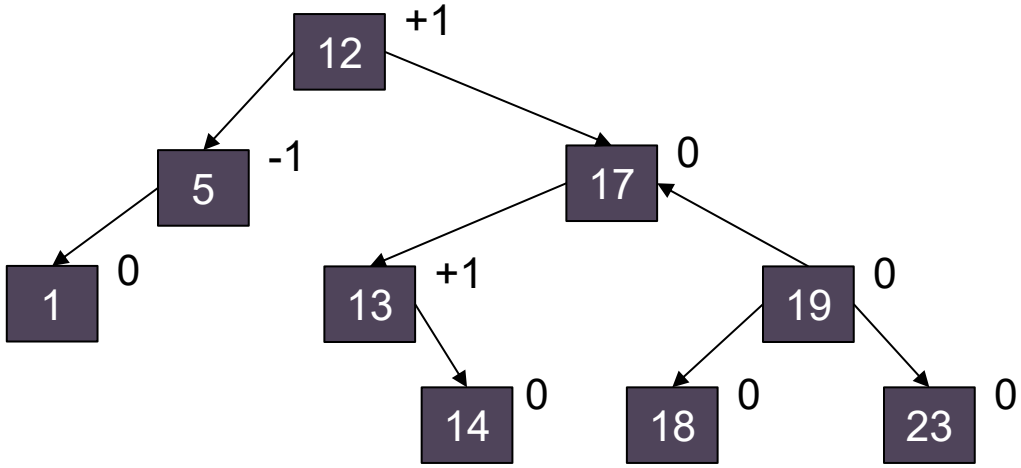

12
+1
5
-1
19
-1
1
0
17
0
23
0
13
0
18
0

AVL-Bäume (14)

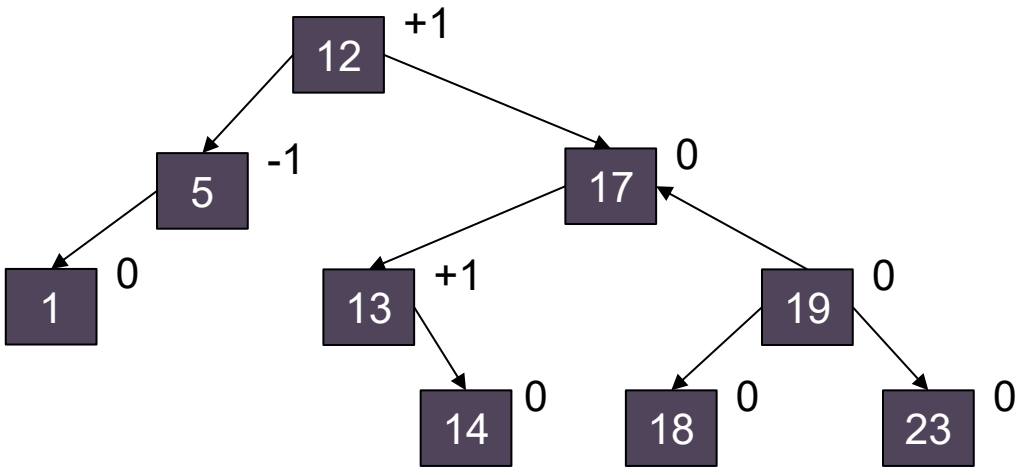


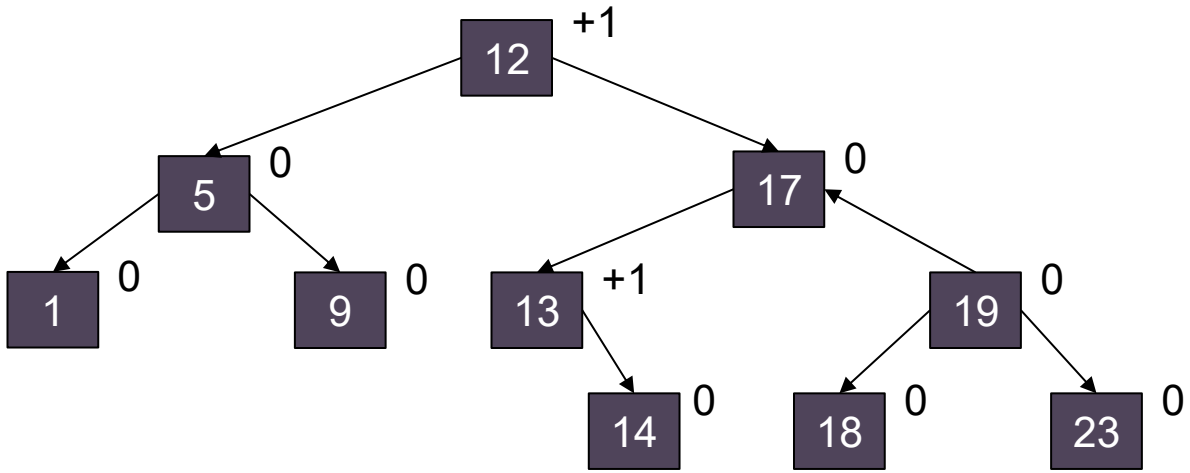


Rechtsrotation

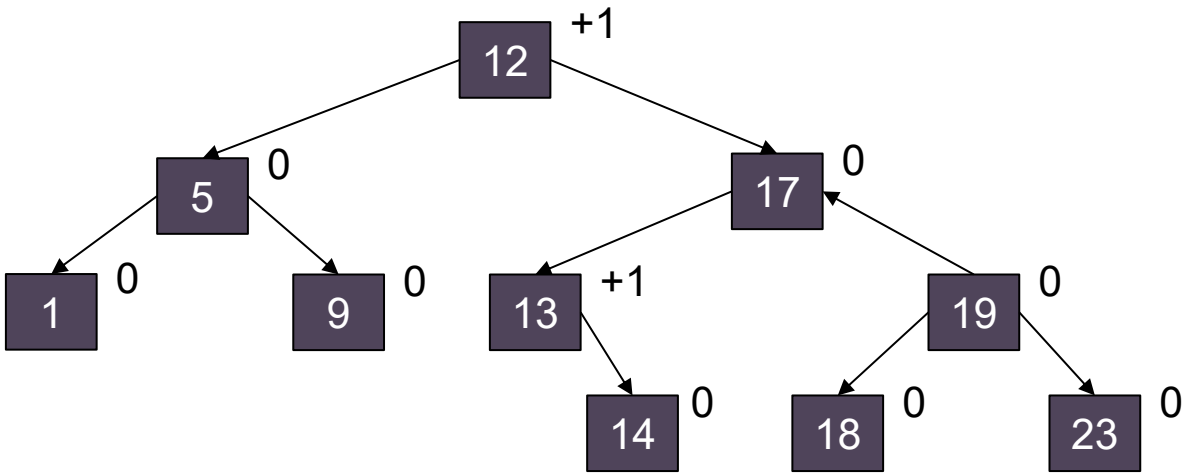


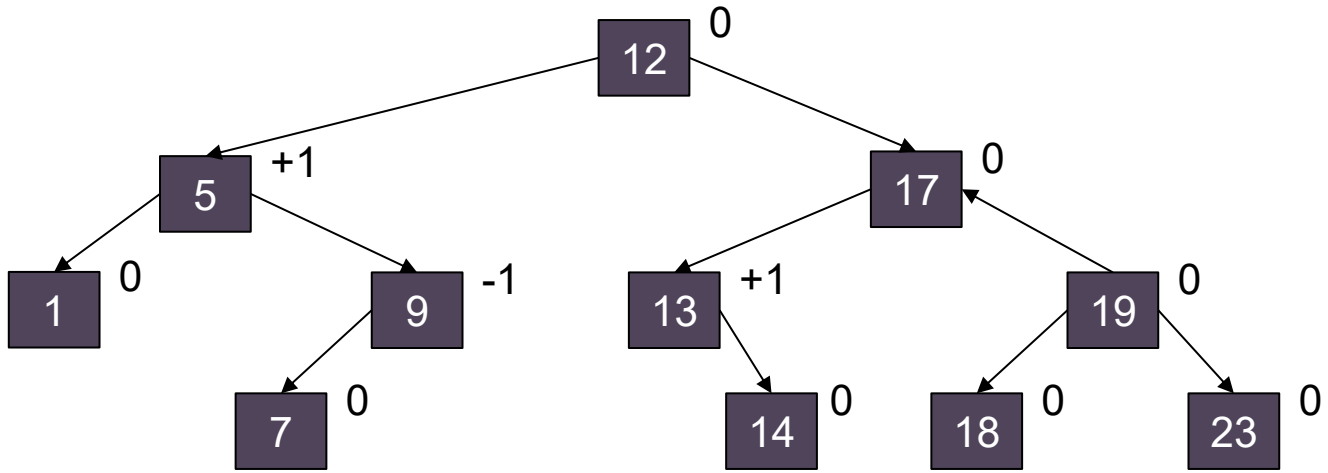
AVL-Bäume (15)


12
+1
5
-1
17
0
1
0
13
+1
19
0
14
0
18
0
23
0

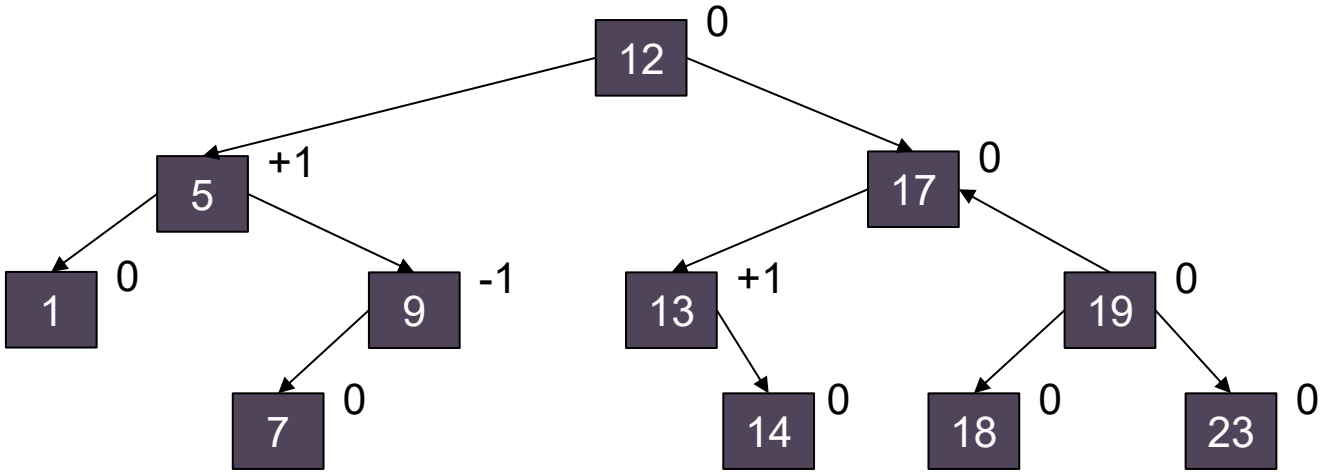

12
+1
5
0
17
0
1
0
9
0
13
+1
19
0
14
0
18
0
23
0

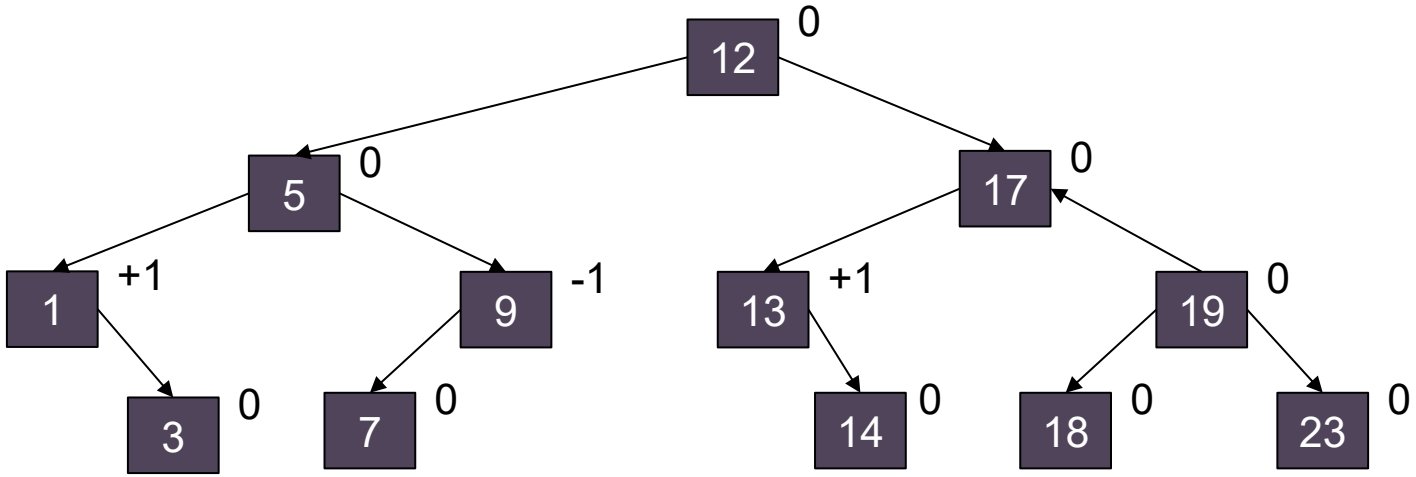
AVL-Bäume (16)


12
+1
5
0
17
0
1
0
9
0
13
+1
19
0
14
0
18
0
23
0

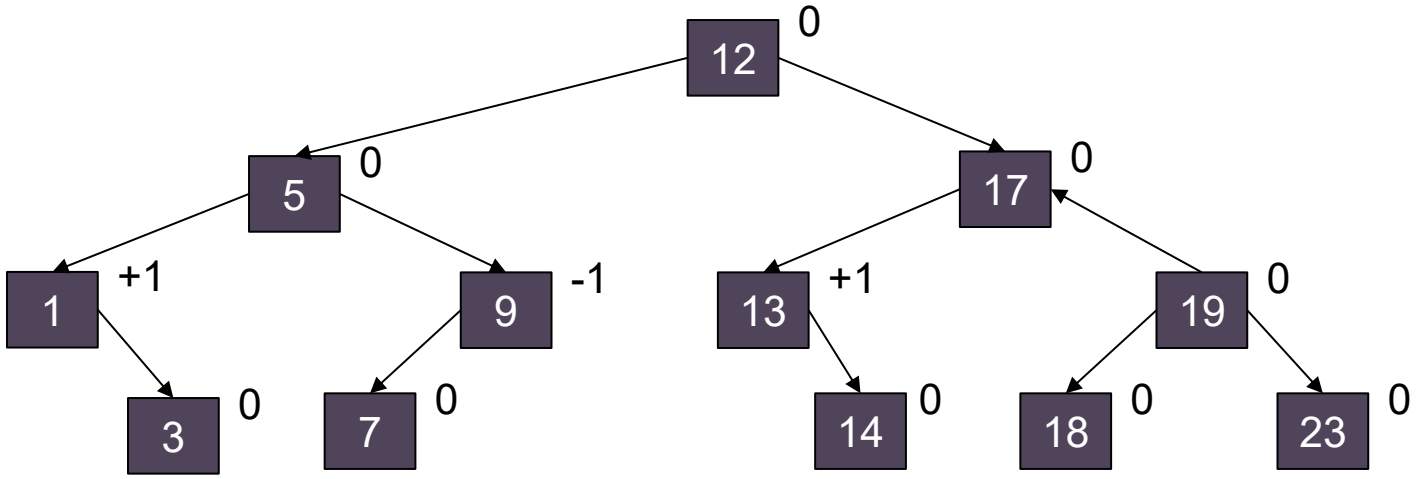

12
0
5
+1
17
0
1
0
9
-1
13
+1
19
0
7
0
14
0
18
0
23
0

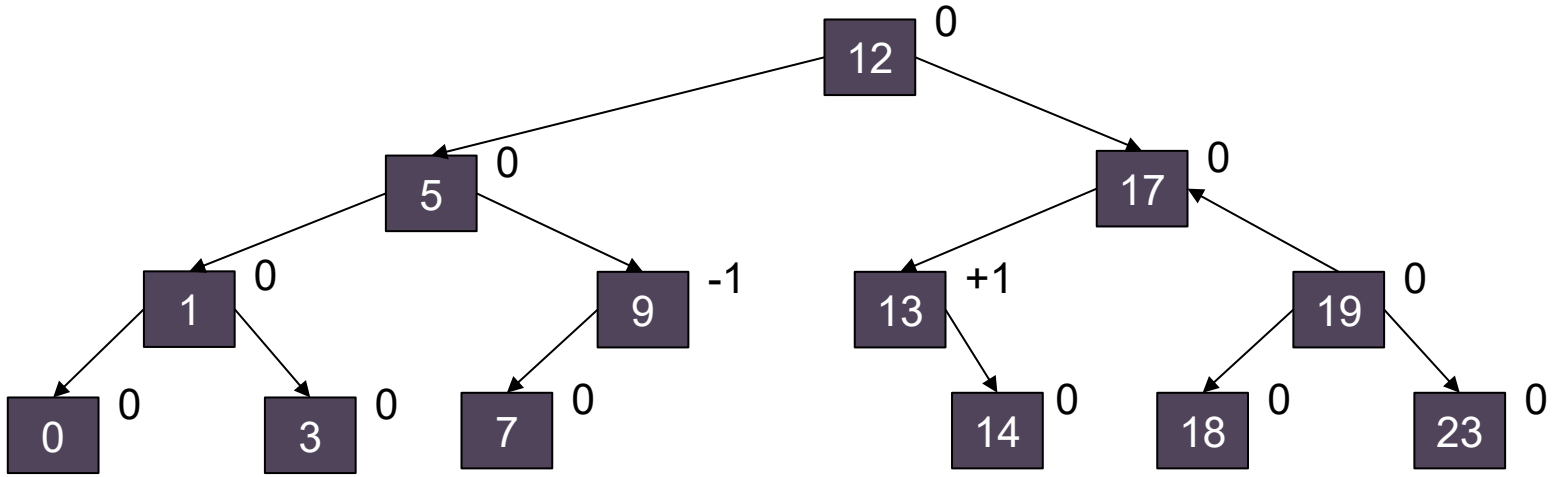
AVL-Bäume (17)


12
0
5
+1
17
0
1
0
9
-1
13
+1
19
0
7
0
14
0
18
0
23
0

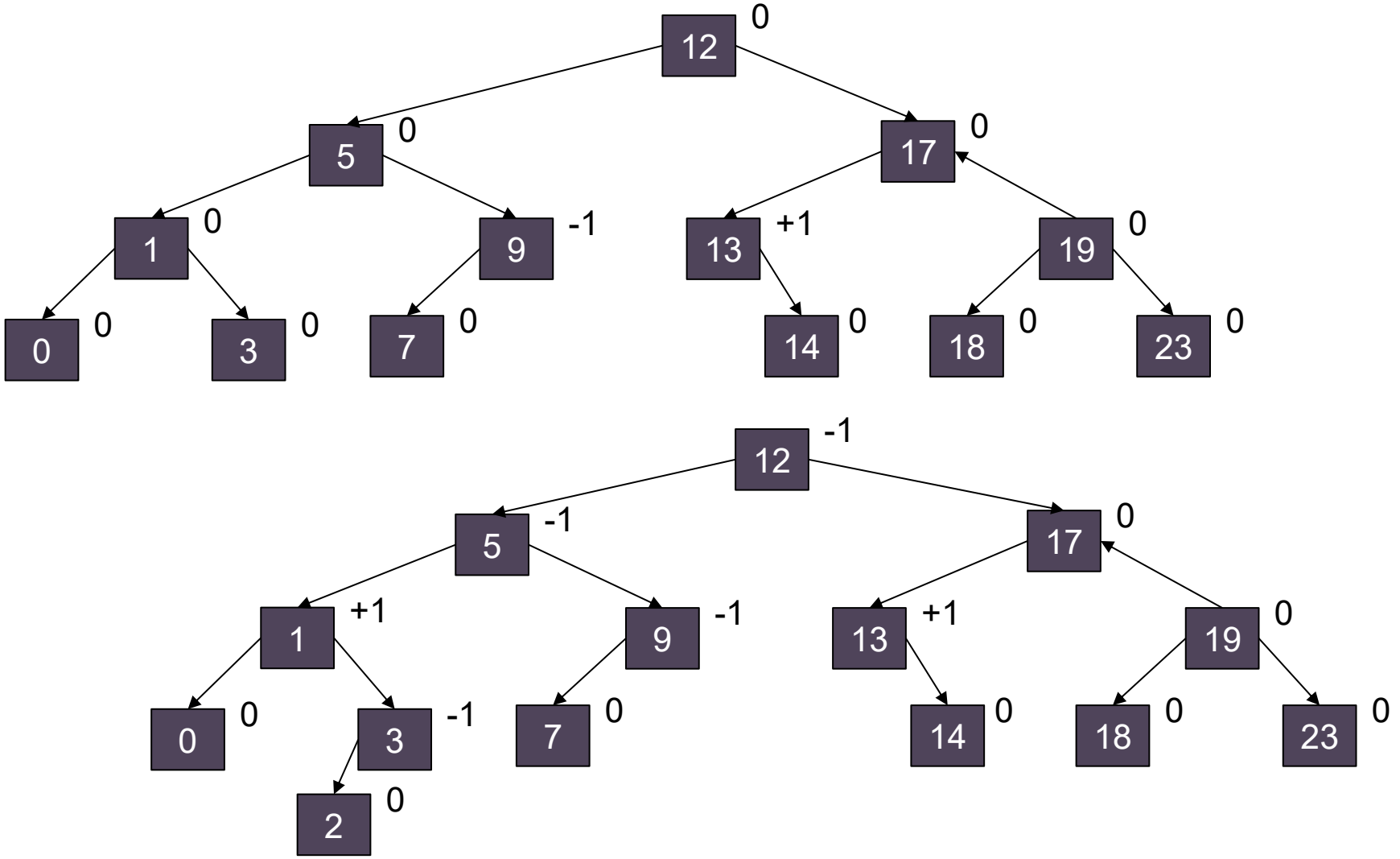

12
0
5
0
17
0
1
+1
9
-1
13
+1
19
0
3
0
7
0
14
0
18
0
23
0

AVL-Bäume (18)

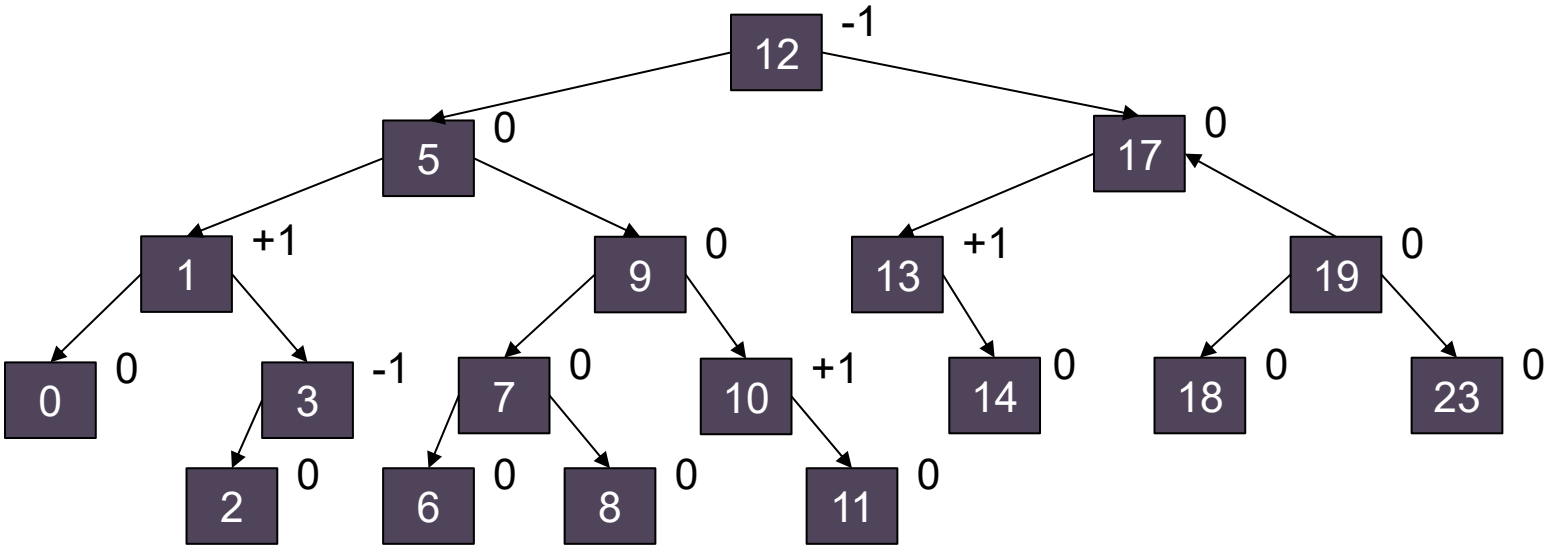

12
0
5
0
17
0
1
+1
9
-1
13
+1
19
0
3
0
7
0
14
0
18
0
23
0


12
0
5
0
17
0
1
0
9
-1
13
+1
19
0
0
0
3
0
7
0
14
0
18
0
23
0

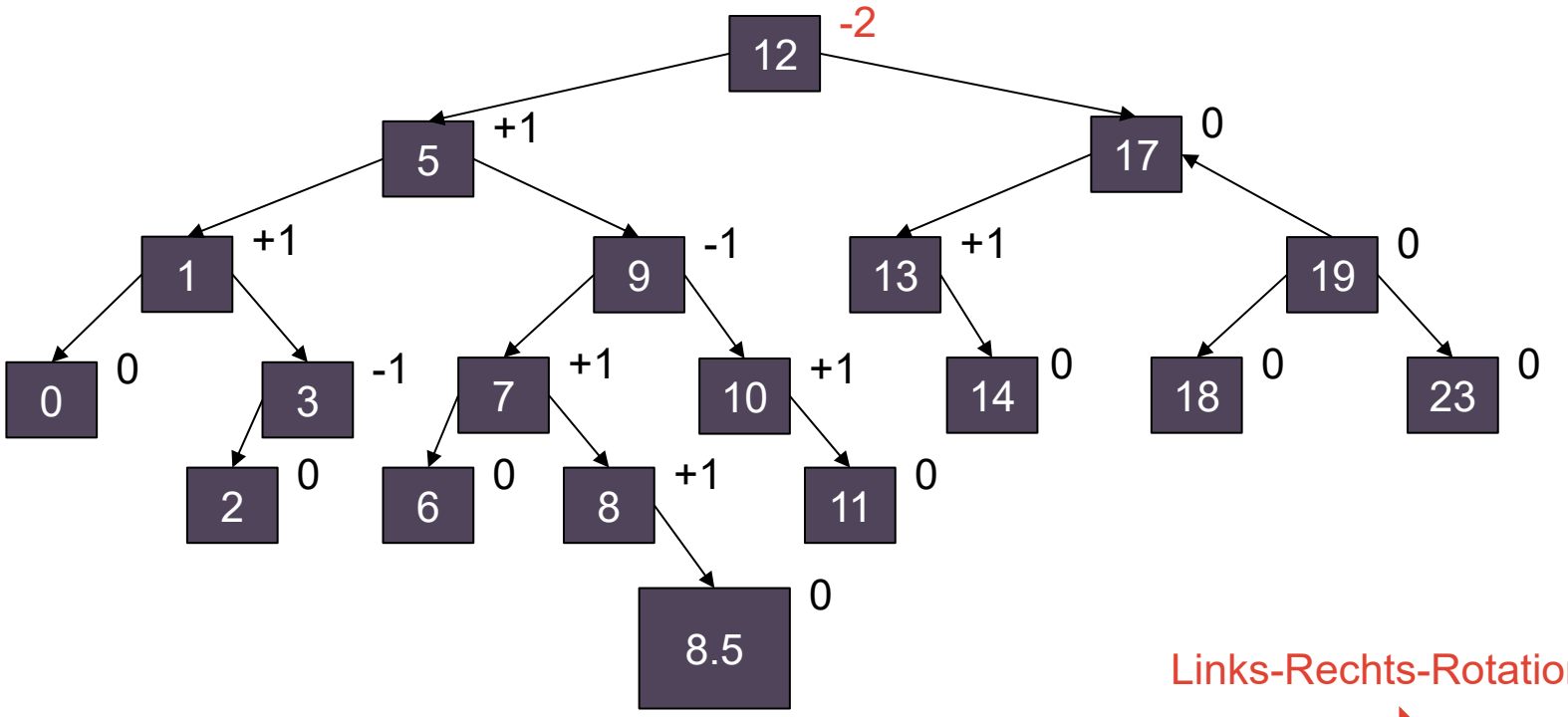
AVL-Bäume (19)


12
0
5
0
17
0
1
0
9
-1
13
+1
19
0
0
0
3
0
7
0
14
0
18
0
23
0
12
-1
5
-1
17
0
1
+1
9
-1
13
+1
19
0
0
0
3
-1
7
0
14
0
18
0
23
0
2
0

AVL-Bäume (20)



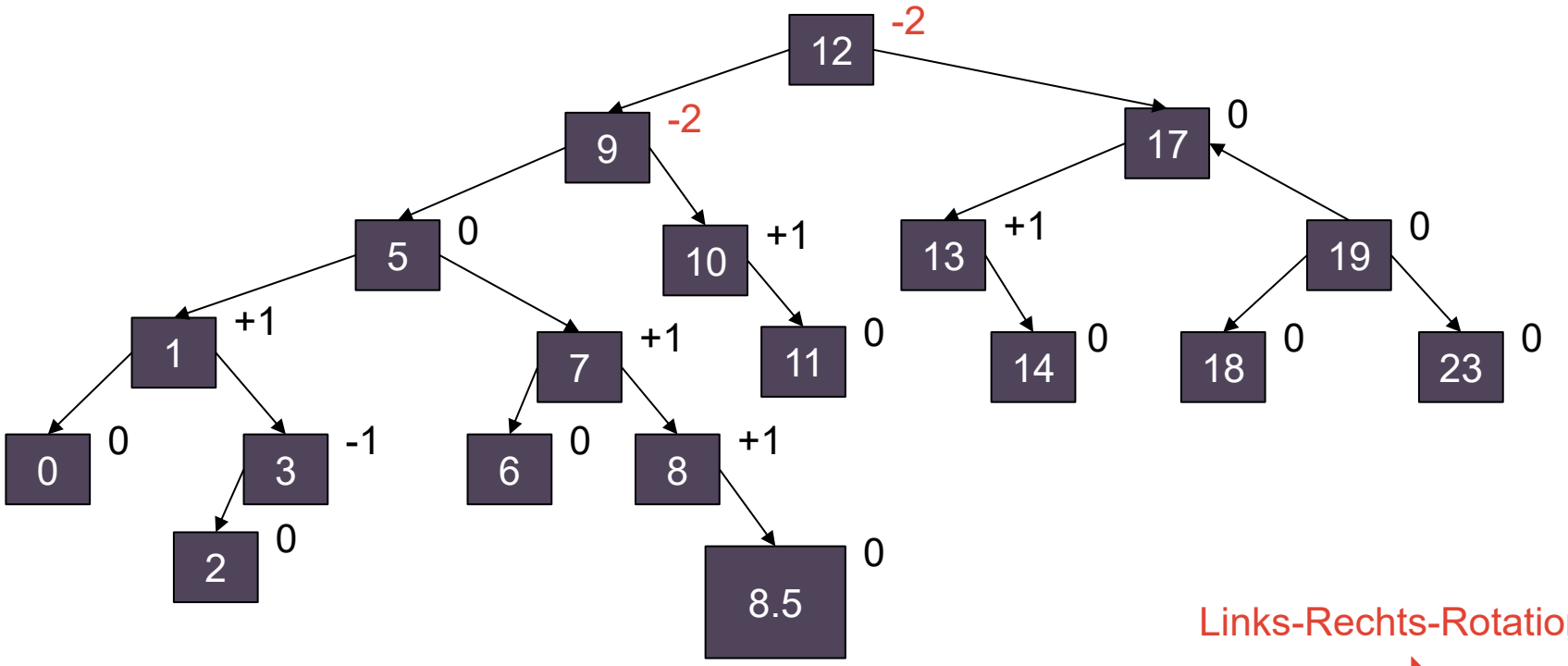
AVL-Bäume (21)



Links-Rechts-Rotation

1. Links-Rotation

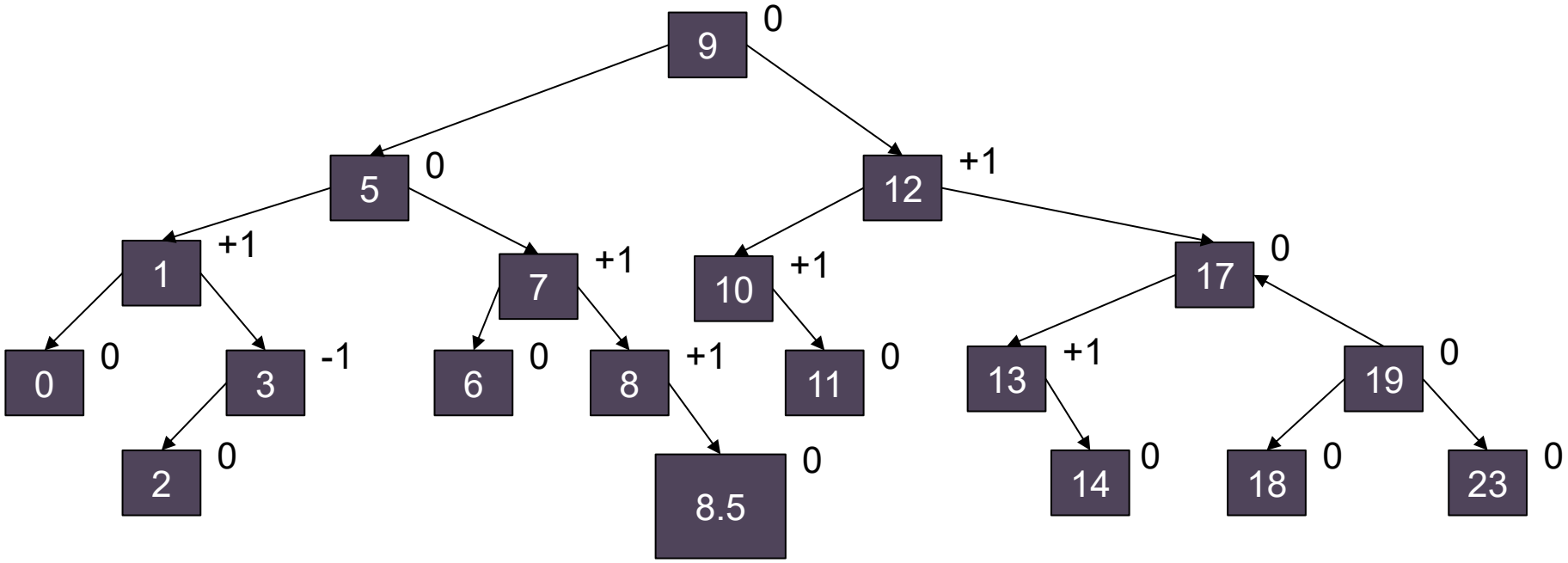
AVL-Bäume (22)



Links-Rechts-Rotation

2. Rechts-Rotation

AVL-Bäume (23)



Vortragsübersicht

7 – Sortierende Container

- Bäume
- Breiten- und Tiefensuche
- AVL-Bäume
- Sets und Maps

Sets und Maps (1)

Grundlagen `set`-Container

- Container `set<K, C, A>` ist sortierte Menge eindeutiger Schlüsselelemente vom Typ `K`
- Header `<set>`

typische Implementierung:

```
template <typename K, typename C, typename A>
    class set
{
    /* ... */
    size_t size_;
    set_node<K> *root_;
    C compare_;                     // Funktor zum Vergleich von Instanzen von K
    allocator_type allocator_;      // über rebind von A für set_node<K>
};

template <typename K>
    class set_node
{
    /* ... */
    set_node *left_, *right_, *parent_;
    K key_;                         // Schlüssel als direkter Member vom Baumknoten
    char balance_factor_;           // Balance-Hilfswert, z. B. Höhendifferenz bei AVL
}
```

Sets und Maps (2)

Wichtige Eigenschaften und Forderungen von `set`-Container

- Elemente liegen jeweils einzeln innerhalb der Baumknoten im Speicher
- Baumknoten liegen (für `std::allocator`) immer auf dem Heap
- Knoten auffindbar über Eltern/Kind-Beziehungen (z. B. Abstieg von der Wurzel)
- Iteratoren sind aus der Bidirectional-Iterator-Kategorie

- Zugriffszeit per gegebenem Wert in O(log *n*)
- Speicherbedarf für *n* Elemente ist (*c* + sizeof(`K`)) * *n* + *d*
  → *c* ist der Bedarf für die Verwaltung einer Heap-Allokation und Balancing
  → *d* ist der Bedarf für den Container selbst
  → Praxis, Rübe nach Arm: *c* ≈ 120 Byte, *d* ≈ 20 Byte

- Vergleich von Schlüsseln über Vergleichsfunktor `compare_` pro Container steuerbar
- üblich: `C == std::less` → `K` muss mit `operator <` das Konzept vergleichbar erfüllen

- Änderungen an der Containergröße invalidieren nur betroffene Elemente!

Sets und Maps (3)

Einfügen und Löschen in `set`-Container

- `std::pair<iterator, bool> set<K, C, A>::insert(const K& key)` (und weitere Überladungen)
- `std::pair<iterator, bool> set<K, C, A>::insert(K&& key)`
  - → Einfügen an richtiger Stelle innerhalb der Ordnung
  - → keine Auswirkungen, falls `key` bezüglich `compare_` schon enthalten
  - → Rückgabe `p.first` ist Iterator auf (neuen oder alten) Knoten mit Element
  - → Rückgabe `p.second` ist `true` falls neuer Knoten, `false` sonst
  - → Laufzeit O(log *n*)
  - → invalidiert keine Iteratoren oder Zeiger

- `iterator set<K, C, A>::erase(set<K, C, A>::iterator pos)` (und weitere Überladungen)
- `iterator set<K, C, A>::erase(set<K, C, A>::const_iterator pos)`
- `size_t set<K, C, A>::erase(const K& key)`
  - → Löschen von Element an Stelle `pos` oder mit Wert `key` (so vorhanden)
  - → Rückgabe: Iterator auf eins nach gelöschtem Element bzw. Zahl gelöschter Elemente
  - → Laufzeit O(log *n*)
  - → invalidiert Iteratoren und Zeiger nur auf das gelöschte Element

Sets und Maps (4)

Suchen in `set`-Container

- `iterator set<K, C, A>::find(const K& key)`
- `const_iterator set<K, C, A>::find(const K& key) const`

  → Suche nach Schlüssel `key` im `set`

  → Rückgabe: Iterator auf Schlüssel, falls vorhanden, sonst `set<K, C, A>::end()`

  → Laufzeit O(log *n*)

- `bool set<K, C, A>::contains(const K& key) const`

  → Abkürzung für `s.find(key) != s.end()`

- `size_t set<K, C, A>::count(const K& key) const`

  → 1, wenn `key` im `set` vorkommt, sonst 0

  → Laufzeit O(log *n*)

  → unterstütze die selbe Schnittstelle wie `multiset`

Sets und Maps (5)

Codebeispiel zu `set`-Container

```
std::set<std::string> s( { "X", "Y", "A", "C", "A", "Y", "B" } );

s.insert("X");                  // ignoriert, "X" schon enthalten
std::cout << s.size();          // Ausgabe: 5, Duplikate werden nicht eingefügt
for (std::string& i : s)
  std::cout << i;          // Ausgabe: A B C X Y, keine Duplikate, automatisch sortiert

std::set<std::string>::iterator ita(s.find("M"));  // schnelle Suche nach Einträgen
if (ita == s.end()) std::cout << "Not found"; // Ausgabe kommt, "M" ist nicht enthalten

std::set<std::string>::iterator itb(s.find("X"));
std::string* itb_ptr = &(*itb);
std::set<std::string>::iterator itc(s.find("Y"));
std::string* itc_ptr = &(*itc);

s.erase("Y");
std::cout << (*itb) << (*itb_ptr);  // OK, Löschen invalidiert nur gelöschtes Element
std::cout << (*itc) << (*itc_ptr);  // undef. VH: Löschen invalidiert alle Zeiger und
                                    // alle Iteratoren auf gelöschte Elemente
```

Sets und Maps (6)

Sortierung eigener Typen im `set`-Container

```
struct part_number
{
    /* ... */
    std::string part; int version;
    bool operator <(const part_number& other) const
    {
        return
            (part < other.part) ||
            ((part == other.part) && (version < other.version));
    }
};

std::set<part_number> s;                        // Reihenfolge nach allen Operationen:
s.insert(part_number("Taster", 17));            //   1) "Bildschirm", 51
s.insert(part_number("Drehknopf", 17));         //   2) "Drehknopf", 17
s.insert(part_number("Taster", 12));            //   3) "Taster", 12
s.insert(part_number("Taster", 17));            //   4) "Taster", 17
s.insert(part_number("Bildschirm", 51));        //   → Duplikat "Taster", 17 fehlt
```

- Angabe eines festen Sortierkriteriums pro Klasse über `operator <`
- Standard-Vergleichsfunktor `std::less` verwendet automatisch diesen `operator <`

Sets und Maps (7)

Sortierung pro Container verändern beim `set`-Container

```
struct part_number
    { /** wie gehabt, inklusive operator < */ };
struct part_number_comparator
{
    bool operator ()
        (const part_number& lhs,
         const part_number& rhs)
    {
        return
            lhs.version > rhs.version;
    }
}

std::set<part_number, part_number_comparator> s;
s.insert(part_number("Blende", 17));              // Reihenfolge nach allen Operationen:
s.insert(part_number("Drehknopf", 17));           //   1) "Halter", 38
s.insert(part_number("Halter", 38));              //   2) "Blende", 17
s.insert(part_number("Mainboard", 17));           //   3) "Blende", 3
s.insert(part_number("Blende", 3));               //   → keine Duplikate für version 17
```

- Sortierkriterium pro Container(-instanz) über `operator ()` in Vergleichsfunktortyp `C`
- Typ `C` kann z. B. auch Membervariablen beinhalten, die das Sortieren steuern
- Achtung: Gleichheit wird vom Sortierkriterium bestimmt! (z. B. auch für `find()`, `erase()`)

Sets und Maps (8)

Negativbeispiel kaputtes Sortierkriterium beim `set`-Container

```
struct part_number
{
    /* ... */
    std::string part; int version;
    bool operator <(const part_number& other) const
    {
        return
            (part < other.part) || (version < other.version);    // keine Totalordnung!
    }
};

part_number a("Blende", 17);   // es gilt: (a < b) == true sowie (b < a) == true,
part_number b("Schraube", 12); // das Problem liegt an operator <, nicht an a und b

std::set<part_number> s;       // Schon ab hier undefiniertes Verhalten!
s.insert(a); s.insert(b);      // genauso bei jedem weiteren Aufruf auf s
```

- Sortierung muss strenge Totalordnung sein: **entweder** `a < b`, **oder** `b < a`, **oder** `a == b`

  → undefiniertes Verhalten, wenn das Sortierkriterium das nicht macht!

  → keine Prüfung auf Semantik durch den Compiler möglich

  → wenn ein kombiniertes Sortierkriterium zu einfach aussieht, ist es vermutlich falsch!

Sets und Maps (9)

Negativbeispiel nachträgliche Änderung der Sortierung beim `set`-Container

```
struct part_number
{
    /* ... */
    std::string part; int version;
    bool operator <(const part_number& other) const
        { return part < other.part }
};

std::set<part_number> s;
s.insert(part_number("Drehknopf", 17));
s.insert(part_number("Blende", 17));
s.insert(part_number("Halter", 17));     // Inhalt: 1) Blende, 2) Drehknopf, 3) Halter

part_number f("Blende", 10);
s.find(f)->version = 23;       // OK, selbst mit Version 10, Sortierung nicht geändert
s.find(f)->part = "Verkleidung";    // undefiniertes Verhalten: Änderung der Sortierung
s.find(f)->part = "Blende";         // z. B. falsches f gefunden, evtl. Duplikate erlaubt
```

- man kann Elemente im set nach dem Einfügen ändern

  → nur Werte ändern, die für die Sortierung nicht relevant sind

  → **sonst: undefiniertes Verhalten!**

Sets und Maps (10)

Grundlagen `map`-Container

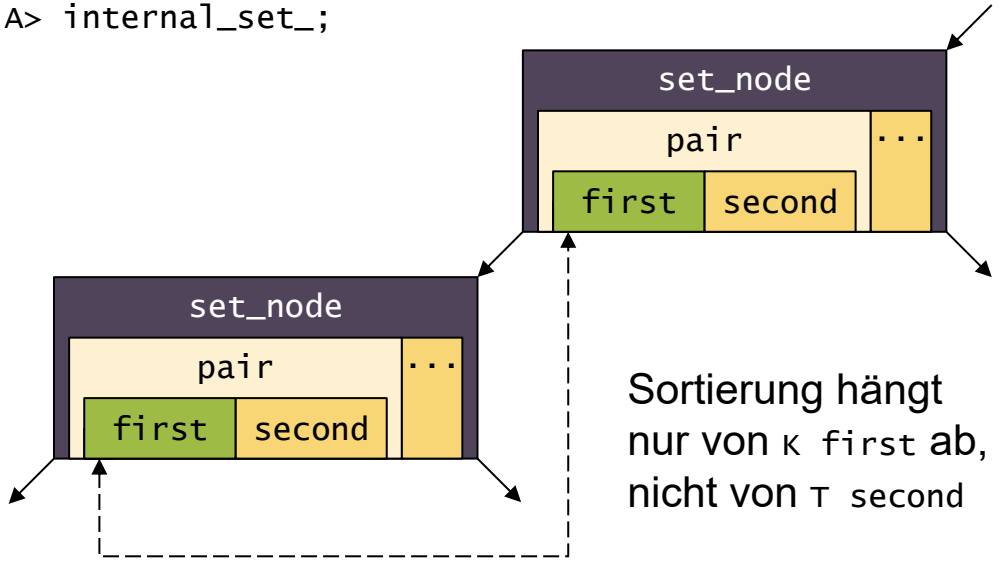
- Container `map<K, T, C, A>` ist Abbildung eindeutiger Schlüsselelemente auf Werte `T`
- Header `<map>`

typische Implementierung:

```
template <typename A, typename B> pair { A first; B second; }
template <typename K, typename T, typename C, typename A> class map
{
    /* ... */
    typedef typename comp_first<C>::type comp_first_with_C;
    set<pair<K, T>, comp_first_with_C, A> internal_set_;
};
```

Grundidee:

`map` ist `set` aus Schlüssel-Wert-Paaren mit Sortierung nur nach Schlüsseln (vgl.: eigenes Sortierkriterium für `set` darf Membervariablen ignorieren)



Sortierung hängt nur von `K first` ab, nicht von `T second`

Sets und Maps (11)

Wichtige Eigenschaften und Forderungen von `map`-Containern

- Funktionen, Eigenschaften, Verhalten (z. B. Iteratoren, Knotenallokation):
  → weitgehend wie ein `std::set<std::pair<K, T>>`
  → Detailänderungen an Funktionen von `set` (z. B. `erase` für `key` statt `pair`)
  → Zusatzfunktionen zur Arbeit mit Paaren (z. B. `operator [ ]` gibt Wert zu Schlüssel)

- Zugriffszeit per gegebenem Schlüssel in O(log *n*)
- Speicherbedarf für *n* Elemente ist (*c* + sizeof(`K`) + sizeof(`T`)) * *n* + *d*
  → *c* ist der Bedarf für die Verwaltung einer Heap-Allokation und Balancing
  → *d* ist der Bedarf für den Container selbst
  → Praxis, Rübe nach Arm: *c* ≈ 120 Byte, *d* ≈ 20 Byte

- Vergleich von Schlüsseln über Vergleichsfunktor `internal_set_.compare_` steuerbar
- üblich: `C == std::less` → `K` muss mit `operator <` das Konzept vergleichbar erfüllen

- Änderungen an der Containergröße invalidieren nur betroffene Elemente!

Sets und Maps (12)

Einfügen von Paaren in `map`-Container

- `std::pair<iterator, bool> map<K, T, C, A>::insert(const pair<K, T>& kv)` (+ Überladungen)
- `std::pair<iterator, bool> map<K, T, C, A>::insert(pair<K, T>&& kv)`
  - → Einfügen an richtiger Stelle innerhalb der Ordnung
  - → Paar in `map` bleibt unverändert, falls `kv.first` bezüglich `compare_` schon enthalten
  - → Rückgabe `p.first` ist Iterator auf (neuen oder alten) Knoten mit Schlüssel-Wert-Paar
  - → Rückgabe `p.second` ist `true` falls neuer Knoten, `false` sonst
  - → Laufzeit O(log *n*)
  - → invalidiert keine Iteratoren oder Zeiger

- `std::pair<iterator, bool> map<K, T, C, A>::insert_or_assign(const pair<K, T>& kv)`
- `std::pair<iterator, bool> map<K, T, C, A>::insert_or_assign(pair<K, T>&& kv)`
  - → grundsätzlich wie `insert()`...
  - → ... aber ersetze bereits vorhandene Schlüssel-Wert-Paare mit `kv`,
    nach Möglichkeit mit Move-Zuweisung, sonst mit Copy-Zuweisung

Sets und Maps (13)

Zugriff auf Einträge in `map`-Container

- `T& map<K, T, C, A>::at(const K& k)`
- `const T& map<K, T, C, A>::at(const K& k) const`

  → sucht nach Paar zu `k` in der `map`

  → gibt Referenz auf Wert aus Paar für k zurück

  → wirft `std::out_of_range`, wenn `k` nicht in `map` enthalten ist

  → Laufzeit O(log *n*)

- `T& map<K, T, C, A>::operator [ ](const K& k)`
- `T& map<K, T, C, A>::operator [ ](K&& k)`

  → sucht nach Paar zu `k` in der `map`

  → gibt Referenz auf Wert aus Paar für `k` zurück

  → falls `k` nicht in `map` ist, erzeuge neues Paar:

  Move- oder Copy-Konstruktor von `k`, Default-Konstruktor für `T`

  → Laufzeit O(log *n*)

Sets und Maps (14)

Codebeispiel zu `map`-Container

```
std::map<std::string, int> m;     // zähle Vorkommen für Bauteile mit Seriennummer

m.insert(std::pair<std::string, int>("Achse", 2));    // Einfügen von Paaren
m.insert(std::make_pair(std::string("Rad"), 4));      // make_pair manchmal kompakter

m.insert(std::pair<std::string, int>("Achse", 4));    // ignoriert, Schlüssel-Duplikat
std::cout << m["Achse"];                              // Zugriff auf Wert im Paar, Ausgabe: 2

m.insert_or_assign                                    // Zuweisung zu bestehendem Paar
     (std::pair<std::string, int>("Achse", 6));
std::cout << m["Achse"];                              // Ausgabe: 6

m["Tür"] = 5;                  // OK, default-konstruierter int, dann überschreiben mit 5
try { m.at("Sitz") = 10; }     // Ausnahme, Schlüssel "Sitz" nicht in m
     catch (...) { }

std::cout << m["Sitz"];        // neuer default-konstruierter Wert, Ausgabe zufälliger int

m.find("Achse")->second = 8;           // OK, Wert ist für Reihenfolge unerheblich
m.find("Achse")->first = "Lager";      // undef. VH, Änderung der Reihenfolge
```

Sets und Maps (15)

Wann nehme ich was?

sortierende Container statt sequentielle Container:

- ich brauche Elemente schnell und automatisch in einer bestimmten Reihenfolge
- ich möchte schnell Duplikate detektieren, aussortieren oder zusammengruppieren
- ich möchte zu gegebenen Werten schnell abhängige Daten finden
- ich habe nicht extrem wenige oder extrem viele Elemente

Set-Container:

- ich möchte schnell und automatisch Duplikate detektieren oder aussortieren
  → Beispiel: Dateinamen von Bilddateien zum Aufräumen der Festplatte

Map-Container:

- ich möchte zu jedem eindeutigen Wert (nicht eindeutige) abhängige Daten speichern
- ich möchte zu gegebenen Werten abhängige Daten schnell finden
  → Beispiel: Zuordnung von Auto-Fahrgestellnummern zu Ausstattungsdetails

Zusammenfassung

Bäume

- Begriffe:
  binäre Bäume, Suchbäume, Suchbaumeigenschaft
- typische Operationen:
  Suchen, Einfügen, Löschen, Traversieren

- Breitensuche: gehe Baum Ebene für Ebene durch
- Tiefensuche: folge (Teil-)Ast bis zum Blatt, ehe weitere (Teil-)Äste geprüft werden

Erweiterungen

- AVL-Bäume verhindern degenerierte Bäume durch Rotationen
- in C++-STL `set` und `map` als Container